

Studentų programa

Generated by Doxygen 1.13.2

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Stud Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 Stud() [1/4]	8
4.1.2.2 Stud() [2/4]	8
4.1.2.3 Stud() [3/4]	9
4.1.2.4 Stud() [4/4]	9
4.1.2.5 ~Stud()	9
4.1.3 Member Function Documentation	9
4.1.3.1 clear()	9
4.1.3.2 galutinis()	9
4.1.3.3 galutinisBalas()	9
4.1.3.4 getEgzaminas()	10
4.1.3.5 getND()	10
4.1.3.6 getPaz()	10
4.1.3.7 grazintiPaskutini()	10
4.1.3.8 operator=() [1/2]	10
4.1.3.9 operator=() [2/2]	10
4.1.3.10 pazKiekis()	10
4.1.3.11 removeLast()	11
4.1.3.12 removeND()	11
4.1.3.13 setBalasGalutinisVid()	11
4.1.3.14 setEgzaminas()	11
4.1.3.15 setND()	11
4.1.3.16 setPavarde()	11
4.1.3.17 setVardas()	11
4.1.4 Friends And Related Symbol Documentation	12
4.1.4.1 operator<<	12
4.1.4.2 operator>>	12
4.2 zmogus Class Reference	12
4.2.1 Detailed Description	13
4.2.2 Constructor & Destructor Documentation	13
4.2.2.1 zmogus() [1/3]	13
4.2.2.2 zmogus() [2/3]	13

4.2.2.3 ~zmogus()	13
4.2.2.4 zmogus() [3/3]	13
4.2.3 Member Function Documentation	13
4.2.3.1 getPavarde()	13
4.2.3.2 getVardas()	14
4.2.3.3 operator=()	14
4.2.3.4 setPavarde()	14
4.2.3.5 setVardas()	14
4.2.4 Member Data Documentation	14
4.2.4.1 pavarde	14
4.2.4.2 vardas	14
5 File Documentation	15
5.1 functions.cpp File Reference	15
5.1.1 Function Documentation	16
5.1.1.1 constrTest()	16
5.1.1.2 copyAssignTest()	16
5.1.1.3 copyConstrTest()	16
5.1.1.4 destructorTest()	16
5.1.1.5 duomenu_generavimas()	16
5.1.1.6 duomenu_ivedimas()	16
5.1.1.7 filePasirinkimas()	16
5.1.1.8 GeneruotiFiles()	17
5.1.1.9 inputTest()	17
5.1.1.10 konstruktoriuTest()	17
5.1.1.11 moveAssignTest()	17
5.1.1.12 moveConstTest()	17
5.1.1.13 operator<<()	17
5.1.1.14 operator>>()	17
5.1.1.15 outputTest()	18
5.1.1.16 PalygintiKategorijas()	18
5.1.1.17 PalygintiPavardes()	18
5.1.1.18 PalygintiVardas()	18
5.1.1.19 paz_gener()	18
5.1.1.20 pazymiu_ivedimas()	18
5.1.1.21 print()	19
5.1.1.22 printCases()	19
5.1.1.23 tikrinimas()	19
5.1.1.24 tinkamas_char()	19
5.1.1.25 tinkamas_int()	19
5.1.1.26 vardai()	19
5.2 functions.cpp	20
5.3 functions.h File Reference	27

5.3.1 Function Documentation	28
5.3.1.1 duomenu_generavimas()	28
5.3.1.2 duomenu_ivedimas()	28
5.3.1.3 filePasirinkimas()	28
5.3.1.4 fileskait()	28
5.3.1.5 GeneruotiFiles()	29
5.3.1.6 konstruktoriuTest()	29
5.3.1.7 PalygintiKategorijas()	29
5.3.1.8 PalygintiPavardes()	29
5.3.1.9 PalygintiVardas()	29
5.3.1.10 paz_gener()	29
5.3.1.11 pazymiu_ivedimas()	30
5.3.1.12 printCases()	30
5.3.1.13 PrintVektorius()	30
5.3.1.14 tinkamas_char()	30
5.3.1.15 tinkamas_int()	30
5.3.1.16 tyrimai()	30
5.3.1.17 vardai()	31
5.3.1.18 vectorIdejimas()	31
5.4 functions.h	31
5.5 main.cpp File Reference	35
5.5.1 Function Documentation	35
5.5.1.1 main()	35
5.6 main.cpp	35
5.7 main.h File Reference	37
5.7.1 Typedef Documentation	38
5.7.1.1 konteinerisDeque	38
5.7.1.2 konteinerisList	38
5.7.1.3 konteinerisVector	38
5.8 main.h	39
5.9 strategija.h File Reference	41
5.9.1 Function Documentation	41
5.9.1.1 nuskaitymasFile()	41
5.9.1.2 studentuIsskirstymas()	42
5.9.1.3 studentuIsskirstymas2()	42
5.9.1.4 studentuRusiavimas()	42
5.9.1.5 studentuRusiavimas2()	42
5.9.1.6 StudentuRusiavimas3()	42
5.9.1.7 studentuSkirstymas3()	43
5.9.1.8 testavimasPrint()	43
5.9.1.9 vykdomaPrograma()	43
5.9.1.10 vykdomaPrograma2()	43

5.9.1.11 vykdomaPrograma3()	43
5.10 strategija.h	44
5.11 Strategija1.cpp File Reference	49
5.11.1 Function Documentation	49
5.11.1.1 main()	49
5.12 Strategija1.cpp	49
5.13 Strategija2.cpp File Reference	50
5.13.1 Function Documentation	50
5.13.1.1 main()	50
5.14 Strategija2.cpp	51
5.15 Strategija3.cpp File Reference	51
5.15.1 Function Documentation	52
5.15.1.1 main()	52
5.16 Strategija3.cpp	52
Index	53

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

zmogus	12
Stud	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Stud	7
zmogus	12

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

functions.cpp	15
functions.h	27
main.cpp	35
main.h	37
strategija.h	41
Strategija1.cpp	49
Strategija2.cpp	50
Strategija3.cpp	51

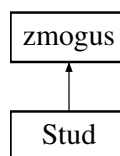
Chapter 4

Class Documentation

4.1 Stud Class Reference

```
#include <main.h>
```

Inheritance diagram for Stud:



Public Member Functions

- [Stud](#) ()
- [Stud](#) (const string &vrđ, const string &pvrđ, int egzam, const vector< int > &namud)
- [Stud](#) (const [Stud](#) &studCopy)
- [Stud](#) & [operator=](#) (const [Stud](#) &studCopy)
- [Stud](#) ([Stud](#) &&studMove)
- [Stud](#) [operator=](#) ([Stud](#) &&studMove)
- void [clear](#) ()
- [~Stud](#) () override
- void [setVardas](#) (const string &var) override
- void [setPavarde](#) (const string &pav) override
- void [setEgzaminas](#) (const int &egz)
- void [setBalasGalutinisVid](#) (const double &Balas)
- void [setND](#) (int paz)
- int [getEgzaminas](#) () const
- int [getPaz](#) (int i) const
- const vector< int > & [getND](#) () const
- double [galutinisBalas](#) () const
- int [pazKiekis](#) () const
- const int & [grazintiPaskutini](#) () const
- void [removeLast](#) ()
- void [removeND](#) ()
- double [galutinis](#) () const

Public Member Functions inherited from [zmogus](#)

- [zmogus](#) ()
- [zmogus](#) (const string &var, const string &pav)
- virtual [~zmogus](#) ()
- string [getVardas](#) () const
- string [getPavarde](#) () const
- [zmogus](#) ([zmogus](#) &&z)
- [zmogus](#) & [operator=](#) ([zmogus](#) &&z)

Friends

- std::ostream & [operator<<](#) (std::ostream &output, const [Stud](#) &studentas)
- std::istream & [operator>>](#) (std::istream &input, [Stud](#) &studentas)

Additional Inherited Members

Protected Attributes inherited from [zmogus](#)

- string [vardas](#)
- string [pavarde](#)

4.1.1 Detailed Description

Definition at line [70](#) of file [main.h](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 [Stud\(\)](#) [1/4]

```
Stud::Stud () [inline]
```

Definition at line [78](#) of file [main.h](#).

4.1.2.2 [Stud\(\)](#) [2/4]

```
Stud::Stud (  
    const string & vrd,  
    const string & pvrđ,  
    int egzam,  
    const vector< int > & namud) [inline]
```

Definition at line [79](#) of file [main.h](#).

4.1.2.3 Stud() [3/4]

```
Stud::Stud (  
    const Stud & studCopy) [inline]
```

copy konstruktorius

Definition at line 88 of file [main.h](#).

4.1.2.4 Stud() [4/4]

```
Stud::Stud (  
    Stud && studMove) [inline]
```

move konstruktorius

Definition at line 110 of file [main.h](#).

4.1.2.5 ~Stud()

```
Stud::~Stud () [inline], [override]
```

Definition at line 134 of file [main.h](#).

4.1.3 Member Function Documentation

4.1.3.1 clear()

```
void Stud::clear () [inline]
```

Definition at line 133 of file [main.h](#).

4.1.3.2 galutinis()

```
double Stud::galutinis () const [inline]
```

galutinio balo skaiciavimo metodas

Definition at line 154 of file [main.h](#).

4.1.3.3 galutinisBalas()

```
double Stud::galutinisBalas () const [inline]
```

Definition at line 147 of file [main.h](#).

4.1.3.4 getEgzaminas()

```
int Stud::getEgzaminas () const [inline]
```

Definition at line 144 of file [main.h](#).

4.1.3.5 getND()

```
const vector< int > & Stud::getND () const [inline]
```

Definition at line 146 of file [main.h](#).

4.1.3.6 getPaz()

```
int Stud::getPaz (  
    int i) const [inline]
```

Definition at line 145 of file [main.h](#).

4.1.3.7 grazintiPaskutini()

```
const int & Stud::grazintiPaskutini () const [inline]
```

Definition at line 149 of file [main.h](#).

4.1.3.8 operator=() [1/2]

```
Stud & Stud::operator= (  
    const Stud & studCopy) [inline]
```

copy assignment operatorius

Definition at line 98 of file [main.h](#).

4.1.3.9 operator=() [2/2]

```
Stud Stud::operator= (  
    Stud && studMove) [inline]
```

move assignment operatorius

Definition at line 118 of file [main.h](#).

4.1.3.10 pazKiekis()

```
int Stud::pazKiekis () const [inline]
```

Definition at line 148 of file [main.h](#).

4.1.3.11 removeLast()

```
void Stud::removeLast () [inline]
```

Definition at line 150 of file [main.h](#).

4.1.3.12 removeND()

```
void Stud::removeND () [inline]
```

Definition at line 151 of file [main.h](#).

4.1.3.13 setBalasGalutinisVid()

```
void Stud::setBalasGalutinisVid (  
    const double & Balas) [inline]
```

Definition at line 140 of file [main.h](#).

4.1.3.14 setEgzaminas()

```
void Stud::setEgzaminas (  
    const int & egz) [inline]
```

Definition at line 139 of file [main.h](#).

4.1.3.15 setND()

```
void Stud::setND (  
    int paz) [inline]
```

Definition at line 141 of file [main.h](#).

4.1.3.16 setPavarde()

```
void Stud::setPavarde (  
    const string & pav) [inline], [override], [virtual]
```

Implements [zmogus](#).

Definition at line 138 of file [main.h](#).

4.1.3.17 setVardas()

```
void Stud::setVardas (  
    const string & var) [inline], [override], [virtual]
```

Implements [zmogus](#).

Definition at line 137 of file [main.h](#).

4.1.4 Friends And Related Symbol Documentation

4.1.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & output,
    const Stud & studentas) [friend]
```

isvedimo operatorius

Definition at line 105 of file [functions.cpp](#).

4.1.4.2 operator>>

```
std::istream & operator>> (
    std::istream & input,
    Stud & studentas) [friend]
```

inputo operatorius

Definition at line 113 of file [functions.cpp](#).

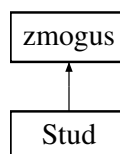
The documentation for this class was generated from the following file:

- [main.h](#)

4.2 zmogus Class Reference

```
#include <main.h>
```

Inheritance diagram for zmogus:



Public Member Functions

- [zmogus](#) ()
- [zmogus](#) (const string &var, const string &pav)
- virtual [~zmogus](#) ()
- virtual void [setVardas](#) (const string &var)=0
- virtual void [setPavarde](#) (const string &pav)=0
- string [getVardas](#) () const
- string [getPavarde](#) () const
- [zmogus](#) (zmogus &&z)
- [zmogus](#) & [operator=](#) (zmogus &&z)

Protected Attributes

- string [vardas](#)
- string [pavarde](#)

4.2.1 Detailed Description

Definition at line [43](#) of file [main.h](#).

4.2.2 Constructor & Destructor Documentation

4.2.2.1 zmogus() [1/3]

```
zmogus::zmogus () [inline]
```

Definition at line [48](#) of file [main.h](#).

4.2.2.2 zmogus() [2/3]

```
zmogus::zmogus (  
    const string & var,  
    const string & pav) [inline]
```

Definition at line [49](#) of file [main.h](#).

4.2.2.3 ~zmogus()

```
virtual zmogus::~zmogus () [inline], [virtual]
```

Definition at line [50](#) of file [main.h](#).

4.2.2.4 zmogus() [3/3]

```
zmogus::zmogus (  
    zmogus && z) [inline]
```

Definition at line [59](#) of file [main.h](#).

4.2.3 Member Function Documentation

4.2.3.1 getPavarde()

```
string zmogus::getPavarde () const [inline]
```

Definition at line [56](#) of file [main.h](#).

4.2.3.2 getVardas()

```
string zmogus::getVardas () const [inline]
```

Definition at line 55 of file [main.h](#).

4.2.3.3 operator=()

```
zmogus & zmogus::operator= (  
    zmogus && z) [inline]
```

Definition at line 62 of file [main.h](#).

4.2.3.4 setPavarde()

```
virtual void zmogus::setPavarde (  
    const string & pav) [pure virtual]
```

Implemented in [Stud](#).

4.2.3.5 setVardas()

```
virtual void zmogus::setVardas (  
    const string & var) [pure virtual]
```

Implemented in [Stud](#).

4.2.4 Member Data Documentation

4.2.4.1 pavarde

```
string zmogus::pavarde [protected]
```

Definition at line 46 of file [main.h](#).

4.2.4.2 vardas

```
string zmogus::vardas [protected]
```

Definition at line 45 of file [main.h](#).

The documentation for this class was generated from the following file:

- [main.h](#)

Chapter 5

File Documentation

5.1 functions.cpp File Reference

```
#include "../functions.h"
```

Functions

- bool [tikrinimas](#) (const [Stud](#) &s, const [Stud](#) &s1)
- void [constrTest](#) ()
- void [copyConstrTest](#) ()
- void [copyAssignTest](#) ()
- void [moveConstTest](#) ()
- void [moveAssignTest](#) ()
- std::ostream & [operator<<](#) (std::ostream &output, const [Stud](#) &studentas)
- std::istream & [operator>>](#) (std::istream &input, [Stud](#) &studentas)
- void [inputTest](#) ()
- void [outputTest](#) ()
- void [destructorTest](#) ()
- void [konstruktoriuTest](#) ()
- bool [tinkamas_char](#) (string vardas)
- bool [tinkamas_int](#) (int skaicius)
- void [pazymiu_ivedimas](#) ([Stud](#) &studentas, string &ss)
- void [paz_gener](#) ([Stud](#) &studentas, string &ss)
- [Stud](#) [vardai](#) ([Stud](#) &studentas, string &ss)
- void [duomenu_ivedimas](#) (vector< [Stud](#) > &studentai, int meniu)
- void [duomenu_generavimas](#) (vector< [Stud](#) > &studentai)
- string [filePasirinkimas](#) ()
- bool [PalygintiVardas](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- bool [PalygintiPavardes](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- bool [PalygintiKategorijas](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- void [printCases](#) (vector< [Stud](#) > visi, bool outputFile, int RusiavimasPagal)
- void [print](#) (vector< [Stud](#) > visi, bool outputFile, int RusiavimasPagal)
- double [GeneruotiFiles](#) (int StudSkaicius)

5.1.1 Function Documentation

5.1.1.1 constrTest()

```
void constrTest ()
```

Definition at line 54 of file [functions.cpp](#).

5.1.1.2 copyAssignTest()

```
void copyAssignTest ()
```

Definition at line 71 of file [functions.cpp](#).

5.1.1.3 copyConstrTest()

```
void copyConstrTest ()
```

Definition at line 61 of file [functions.cpp](#).

5.1.1.4 destructorTest()

```
void destructorTest ()
```

Definition at line 147 of file [functions.cpp](#).

5.1.1.5 duomenu_generavimas()

```
void duomenu_generavimas (  
    vector< Stud > & studentai)
```

Definition at line 429 of file [functions.cpp](#).

5.1.1.6 duomenu_ivedimas()

```
void duomenu_ivedimas (  
    vector< Stud > & studentai,  
    int menu)
```

Definition at line 329 of file [functions.cpp](#).

5.1.1.7 filePasirinkimas()

```
string filePasirinkimas ()
```

Definition at line 445 of file [functions.cpp](#).

5.1.1.8 GeneruotiFiles()

```
double GeneruotiFiles (  
    int StudSkaicius)
```

Definition at line 602 of file [functions.cpp](#).

5.1.1.9 inputTest()

```
void inputTest ()
```

Definition at line 126 of file [functions.cpp](#).

5.1.1.10 konstruktoriuTest()

```
void konstruktoriuTest ()
```

Definition at line 156 of file [functions.cpp](#).

5.1.1.11 moveAssignTest()

```
void moveAssignTest ()
```

Definition at line 93 of file [functions.cpp](#).

5.1.1.12 moveConstTest()

```
void moveConstTest ()
```

Definition at line 81 of file [functions.cpp](#).

5.1.1.13 operator<<()

```
std::ostream & operator<< (  
    std::ostream & output,  
    const Stud & studentas)
```

isvedimo operatorius

Definition at line 105 of file [functions.cpp](#).

5.1.1.14 operator>>()

```
std::istream & operator>> (  
    std::istream & input,  
    Stud & studentas)
```

inputo operatorius

Definition at line 113 of file [functions.cpp](#).

5.1.1.15 outputTest()

```
void outputTest ()
```

Definition at line 137 of file [functions.cpp](#).

5.1.1.16 PalygintiKategorijas()

```
bool PalygintiKategorijas (  
    const Stud & stud1,  
    const Stud & stud2)
```

Definition at line 513 of file [functions.cpp](#).

5.1.1.17 PalygintiPavardes()

```
bool PalygintiPavardes (  
    const Stud & stud1,  
    const Stud & stud2)
```

Definition at line 508 of file [functions.cpp](#).

5.1.1.18 PalygintiVardas()

```
bool PalygintiVardas (  
    const Stud & stud1,  
    const Stud & stud2)
```

Definition at line 503 of file [functions.cpp](#).

5.1.1.19 paz_gener()

```
void paz_gener (  
    Stud & studentas,  
    string & ss)
```

Definition at line 274 of file [functions.cpp](#).

5.1.1.20 pazymiu_ivedimas()

```
void pazymiu_ivedimas (  
    Stud & studentas,  
    string & ss)
```

Definition at line 235 of file [functions.cpp](#).

5.1.1.21 print()

```
void print (  
    vector< Stud > visi,  
    bool outputFile,  
    int RusiavimasPagal)
```

Definition at line 572 of file [functions.cpp](#).

5.1.1.22 printCases()

```
void printCases (  
    vector< Stud > visi,  
    bool outputFile,  
    int RusiavimasPagal)
```

Definition at line 517 of file [functions.cpp](#).

5.1.1.23 tikrinimas()

```
bool tikrinimas (  
    const Stud & s,  
    const Stud & s1)
```

Definition at line 46 of file [functions.cpp](#).

5.1.1.24 tinkamas_char()

```
bool tinkamas_char (  
    string vardas)
```

Definition at line 168 of file [functions.cpp](#).

5.1.1.25 tinkamas_int()

```
bool tinkamas_int (  
    int skaicius)
```

Definition at line 180 of file [functions.cpp](#).

5.1.1.26 vardai()

```
Stud vardai (  
    Stud & studentas,  
    string & ss)
```

Definition at line 285 of file [functions.cpp](#).

5.2 functions.cpp

[Go to the documentation of this file.](#)

```

00001 #include "../functions.h"
00002
00003 //realizuotas copy konstruktorius
00004 /*Stud::Stud(const Stud& studCopy) :
00005     /*vardas(studCopy.vardas),
00006     pavarde(studCopy.pavarde), uzkommentint
00007     zmogus(studCopy.getVardas(), studCopy.getPavarde()),
00008     egzaminas(studCopy.egzaminas),
00009     nd(studCopy.nd),
00010     BalasGalutinisVid(studCopy.BalasGalutinisVid),
00011     balasSuskaiciuotas(studCopy.balasSuskaiciuotas) {*/
00012
00013 //realizuotas copy assignment operatorius
00014 /*Stud& Stud::operator=(const Stud& studCopy){
00015     if(this != &studCopy){
00016         setVardas(studCopy.getVardas());
00017         setPavarde(studCopy.getPavarde());
00018         nd = studCopy.nd;
00019         egzaminas = studCopy.egzaminas;
00020         BalasGalutinisVid = studCopy.BalasGalutinisVid;
00021         balasSuskaiciuotas = studCopy.balasSuskaiciuotas;}
00022     return *this;
00023 }*/
00024
00025 //realizuotas move konstruktorius
00026 /*Stud::Stud(Stud&& studMove) :           //normalu destruktoriau pasidaryti ji iskviesti ir tada tikrinti ar yra lygus 0
00027
00028     zmogus(std::move(studMove)),
00029     nd(std::move(studMove.nd)),
00030     egzaminas(std::move(studMove.egzaminas)),
00031     BalasGalutinisVid(std::move(studMove.BalasGalutinisVid)),
00032     balasSuskaiciuotas(std::move(studMove.balasSuskaiciuotas)){studMove.clear();}*/
00033
00034 //realizuotas move assignment operatorius
00035 /*Stud& Stud::operator=(Stud&& studMove){
00036     if (this != &studMove) {
00037         zmogus::operator=(std::move(studMove));
00038         nd = std::move(studMove.nd);
00039         egzaminas = std::move(studMove.egzaminas);
00040         BalasGalutinisVid = std::move(studMove.BalasGalutinisVid);
00041         balasSuskaiciuotas = std::move(studMove.balasSuskaiciuotas);
00042         studMove.clear();
00043     }
00044     return *this;}*/
00045
00046 bool tikrinimas(const Stud& s, const Stud& s1){
00047     if(s.getVardas() == s1.getVardas() && s.getPavarde() == s1.getPavarde() && s.getEgzaminas() == s1.getEgzaminas()
00048     && s.getND() == s1.getND()){
00049         return true;
00050     }
00051     else return false;
00052 }
00053 //default constructor test
00054 void constrTest(){
00055     Stud s, s1;
00056     if(tikrinimas(s, s1)){cout << std::left << std::setw(31) << "Default konstruktorius" << "| +\n";}
00057     else cout << std::left << std::setw(31) << "Default konstruktorius" << "| -\n";
00058 }
00059
00060 //copy constructor test
00061 void copyConstrTest(){
00062     Stud s("Astrida", "Jablonskyté", 10, {7, 7, 7});
00063     Stud s1(s); //copy
00064     if(tikrinimas(s1, s)){
00065         s.setVardas("Amelija"); //testuojama ar kopijos reiksme nepasikeite pakeitus originala
00066         if(!tikrinimas(s1, s)){cout << std::left << std::setw(31) << "Copy konstruktorius" << "| +\n";}
00067         else cout << std::left << std::setw(31) << "Copy konstruktorius" << "| -\n";
00068     }
00069
00070 //copy assignment test
00071 void copyAssignTest(){
00072     Stud s1("Monika", "Mocevičiute", 9, {8, 8, 8}), s("Astrida", "Jablonskyté", 10, {7, 7, 7});
00073     s1 = s; //iskvieciamas copy assignment konstruktorius
00074     if(tikrinimas(s1, s)){
00075         s.setVardas("Amelija"); //testuojama ar kopijos reiksme nepasikeite pakeitus originala
00076         if(!tikrinimas(s1, s)){cout << "Copy assignment operatorius | +\n";}
00077         else cout << "Copy assignment operatorius | -\n";
00078     }
00079
00080 //move konstruktorius test
00081 void moveConstrTest(){

```

```

00082     Stud s("Astrida", "Jablonskyte", 10, {7, 7, 7});
00083     Stud s1(std::move(s));
00084     s.~Stud();
00085     if(s1.getVardas() == "Astrida" && s1.getPavarde() == "Jablonskyte" && s1.getEgzaminas() == 10 && s1.getND() ==
vector<int>{7,7,7}){
00086         if(s.getVardas().empty() && s.getPavarde().empty() && s.getND().empty() && s.getEgzaminas() == 0){
00087             cout << std::left << std::setw(31) << "Move konstruktorius" << "| +\n";
00088         }else cout << std::left << std::setw(31) << "Move konstruktorius" << "| -\n";
00089     }else cout << std::left << std::setw(31) << "Move konstruktorius" << "| -\n";
00090 }
00091
00092 //move assignment test
00093 void moveAssignTest(){
00094     Stud s1("Monika", "Mocevičiute", 9, {8, 8, 8}), s("Astrida", "Jablonskyte", 10, {7, 7, 7});
00095     s1 = std::move(s);
00096     s.~Stud();
00097     if(s1.getVardas() == "Astrida" && s1.getPavarde() == "Jablonskyte" && s1.getEgzaminas() == 10 && s1.getND() ==
vector<int>{7,7,7}){
00098         if(s.getVardas().empty() && s.getPavarde().empty() && s.getND().empty() && s.getEgzaminas() == 0){
00099             cout << "Move assignment operatorius" << "| +\n";
00100         }else cout << "Move assignment operatorius" << "| -\n";
00101     }else cout << "Move assignment operatorius" << "| -\n";
00102 }
00103
00105 std::ostream& operator<(std::ostream &output, const Stud& studentas){
00106     output << std::setw(15) << std::left << studentas.pavarde
00107     << std::setw(15) << std::left << studentas.vardas
00108     << std::setw(15) << std::fixed << std::setprecision(2) << studentas.galutinis() << endl;
00109     return output;
00110 }
00111
00113 std::istream& operator>(std::istream &input, Stud& studentas){ //naudoti ifstreama
00114     int paz;
00115     input > studentas.vardas > studentas.pavarde;
00116     while(input > paz){
00117         studentas.nd.push_back(paz);}
00118
00119     studentas.egzaminas = studentas.nd.back();
00120     studentas.nd.pop_back();
00121     studentas.galutinis();
00122     return input;
00123 }
00124
00125 //inputo testas
00126 void inputTest(){
00127     Stud stud;
00128     vector<int> nd{7,8,9};
00129     std::istringstream ss("Astrida Jablonskyte 7 8 9 10");
00130     ss>stud;
00131     if(stud.getVardas() == "Astrida" && stud.getPavarde() == "Jablonskyte" && stud.getND() == nd &&
stud.getEgzaminas() == 10)
00132     {cout << std::left << std::setw(31) << "Ivesties operatorius" << "| +\n"; }
00133     else cout << std::left << std::setw(31) << "Ivesties operatorius" << "| -\n";
00134 }
00135
00136 //outputo testas
00137 void outputTest(){
00138     Stud studentas("Astrida", "Jablonskyte", 10, {10, 10, 10});
00139     std::stringstream outputTestas, output;
00140
00141     outputTestas<<std::setw(15)<<std::left<<"Jablonskyte"<<std::setw(15)<<std::left<<"Astrida"<<std::setw(15)<<std::fixed<<std::setprecision(2)<<10.00
00142     output<<studentas;
00143     if(outputTestas.str() == output.str()){cout << std::left << std::setw(31) << "Ivesties operatorius" << "| +\n";}
00144     else cout << std::left << std::setw(31) << "Ivesties operatorius" << "| -\n";
00145 }
00146 //destruktoriaus testas
00147 void destructorTest(){
00148     Stud* studentas = new Stud();
00149     for(int i = 2; i < 7; i++){studentas->setND(i);}
00150     studentas->~Stud();
00151     if (studentas->getND().empty()){cout << std::left << std::setw(31) << "Dstruktorius" << "| +\n";}
00152     else cout << std::left << std::setw(31) << "Dstruktorius" << "| -\n";
00153     operator delete(studentas);
00154 }
00155
00156 void konstruktoriuTest(){
00157     cout << "... \n";
00158     constrTest();
00159     copyConstrTest();
00160     copyAssignTest();
00161     moveConstrTest();
00162     moveAssignTest();
00163     inputTest();
00164     outputTest();
00165     destructorTest();
00166 }

```

```

00167
00168 bool tinkamas_char(string vardas)
00169 {
00170     for (char a : vardas)
00171     {
00172         if (isalpha(a) == false || isspace(a) == true)
00173         {
00174             return false;
00175         }
00176     }
00177     return true;
00178 }
00179
00180 bool tinkamas_int(int skaicius)
00181 {
00182     while (skaicius > 10 || skaicius < 0)
00183     {
00184         if (skaicius == 0)
00185         {
00186             break;
00187         }
00188         return false;
00189     }
00190     return true;
00191 }
00192
00193 /*double galutinis(Stud studentas, int PagalSkaiciavimas)
00194 {
00195     if (PagalSkaiciavimas == 0)
00196     {
00197         double med;
00198         if (studentas.nd.size() == 1)
00199         {
00200             med = studentas.nd[0];
00201         }
00202
00203         else if (studentas.nd.size() % 2 == 0)
00204         {
00205             sort(studentas.nd.begin(), studentas.nd.end());
00206             med = (studentas.nd[studentas.nd.size() / 2 - 1] + studentas.nd[studentas.nd.size() / 2]) / 2;
00207         }
00208         else
00209         {
00210             med = studentas.nd[round(studentas.nd.size() / 2) - 1];
00211         }
00212         return round((0.4 * med + 0.6 * studentas.egzaminas) * 100) / 100;
00213     }
00214     else
00215     {
00216         double vidurkis;
00217         double suma{0.0};
00218         if (studentas.nd.size() == 1)
00219         {
00220             vidurkis = studentas.nd[0];
00221         }
00222         else
00223         {
00224             for (size_t ss = 0; ss < studentas.nd.size(); ss++)
00225             {
00226                 suma += studentas.nd[ss];
00227             }
00228
00229             vidurkis = suma / studentas.nd.size();
00230         }
00231         return round((0.4 * vidurkis + 0.6 * studentas.egzaminas) * 100) / 100;
00232     }
00233 }*/
00234
00235 void pazymiu_ivedimas(Stud &studentas, string& ss){
00236     studentas.removeND();
00237     bool stop1 = false;
00238     cout << "Iveskite studento pazymius (-i), baige iveskite 0" << endl;
00239     do
00240     {
00241         while (true)
00242         {
00243             try
00244             {
00245                 int paz;
00246                 cin>>paz;
00247                 if (cin.fail())
00248                 {
00249                     cin.clear();
00250                     cin.ignore(numeric_limits<streamsize>::max(), '\n');
00251                     throw "Neteisingas ivestas skaicius";
00252                 }
00253                 if (tinkamas_int(paz) == false)

```

```

00254         {
00255             throw "Neteisingai ivestas skaicius";
00256         }
00257         if (paz == 0)
00258         {
00259             stop1 = true;
00260         }
00261         else
00262             ss += std::to_string(paz) + " ";
00263         break;
00264     }
00265     catch (const char *masyvas)
00266     {
00267         cout << masyvas << endl;
00268     }
00269 }
00270
00271 } while (!stop1);
00272 }
00273
00274 void paz_gener(Stud &studentas, string& ss)
00275 {
00276     int A = rand() % 10 + 2;
00277     for (int i = 0; i < A; i++)
00278     {
00279         int P = rand() % 10 + 1;
00280         ss += std::to_string(P) + " ";
00281     }
00282     cout << endl;
00283 }
00284
00285 Stud vardai(Stud &studentas, string& ss)
00286 {
00287     int vsuma = 0;
00288     string F, m, n;
00289
00290     int lytis = rand() % 100 + 1;
00291     vector<string> Vmasyvas;
00292     vector<string> Pmasyvas;
00293     ifstream f("vardai_moteru.txt");
00294     ifstream l("vardai_vyru.txt");
00295     while (getline(f, F))
00296     {
00297         vsuma += 1;
00298     }
00299     int C = rand() % vsuma;
00300     int W = rand() % vsuma;
00301     f.clear();
00302     f.seekg(0, ios::beg);
00303     if (lytis % 2 == 0)
00304     {
00305         for (int i = 0; i < vsuma; i++)
00306         {
00307             f >> m >> n;
00308             Vmasyvas.push_back(m);
00309             Pmasyvas.push_back(n);
00310         }
00311         f.close();
00312         ss += Vmasyvas[C] + " " + Pmasyvas[W] + " ";
00313         return studentas;
00314     }
00315     else
00316     {
00317         for (int y = 0; y < vsuma; y++)
00318         {
00319             l >> m >> n;
00320             Vmasyvas.push_back(m);
00321             Pmasyvas.push_back(n);
00322         }
00323         l.close();
00324         ss += Vmasyvas[C] + " " + Pmasyvas[W] + " ";
00325         return studentas;
00326     }
00327 }
00328
00329 void duomenu_ivedimas(vector<Stud> &studentai, int meniu)
00330 {
00331     string fullLine;
00332     int egzaminas;
00333     bool stop;
00334     cin.ignore(numeric_limits<streamsize>::max(), '\n');
00335     string vardas, pavarde;
00336     do
00337     {
00338         Stud studentas;
00339         if (menu == 1 || menu == 2)

```

```

00341     {
00342         cout << "Iveskite studento vardą, baigę iveskite 'stop'" << endl;
00343     }
00344     while (true)
00345     {
00346         try
00347         {
00348             getline(cin, vardas);
00349             if (tinkamas_char(vardas) == false)
00350             {
00351                 cin.clear();
00352                 throw "Neteisingai įvedėte studento vardą";
00353             }
00354             break;
00355         }
00356         catch (const char *masyvas)
00357         {
00358             cout << masyvas << endl;
00359         }
00360     }
00361     if (vardas == "stop"){
00362         stop = true;
00363         break;
00364     }
00365     cout << "Iveskite studento pavardę" << endl;
00366     while (true)
00367     {
00368         try
00369         {
00370             getline(cin, pavarde);
00371             if (tinkamas_char(pavarde) == false)
00372             {
00373                 cin.clear();
00374                 throw "Neteisingai įvedėte studento pavardę";
00375             }
00376             break;
00377         }
00378         catch (const char *masyvas)
00379         {
00380             cout << masyvas << endl;
00381         }
00382     }
00383     cout << "Iveskite egzaminų balą" << endl;
00384     while (true)
00385     {
00386         try
00387         {
00388             cin>>egzaminas;
00389             if (cin.fail())
00390             {
00391                 cin.clear();
00392                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00393                 throw "Iveskite skaičių";
00394             }
00395             if (tinkamas_int(egzaminas) == false)
00396             {
00397                 throw "Iveskite teisingą pažymį";
00398             }
00399             break;
00400         }
00401         catch (const char *masyvas)
00402         {
00403             cout << masyvas << endl;
00404         }
00405     }
00406 }
00407
00408     fullLine = vardas + " " + pavarde + " ";
00409 }
00410 if (menu == 1)
00411 {
00412     pazymiu_ivedimas(studentas, fullLine);
00413 }
00414 else if (menu == 2)
00415 {
00416     paz_gener(studentas, fullLine);
00417 }
00418
00419     fullLine += std::to_string(egzaminas) + " ";
00420
00421     std::istringstream ss(fullLine);
00422     ss>>studentas;
00423     studentai.push_back(studentas);
00424     cout << "..." << endl;
00425     cin.ignore(numeric_limits<streamsize>::max(), '\n');
00426 } while (!stop);
00427 }

```

```

00428
00429 void duomenu_generavimas(vector<Stud> &studentai)
00430 {
00431     int Q = rand() % 10 + 1;
00432     for (int i = 0; i < Q; i++)
00433     {string fullLine;
00434         Stud studentas;
00435         vardai(studentas, fullLine);
00436         paz_gener(studentas, fullLine);
00437         fullLine += std::to_string(rand() % 10 + 1);
00438         std::stringstream ss(fullLine);
00439         cout < fullLine;
00440         ss>studentas;
00441         studentai.push_back(studentas);
00442     }
00443 }
00444
00445 string filePasirinkimas()
00446 {
00447     system("for /r %A in (*.txt) do @echo %~nxA > temp.txt");
00448     ifstream f;
00449     string eilute;
00450     int pasirinkimuSUM = 0;
00451     int pasirinkimas;
00452     vector<string> txtfiles;
00453     vector<string> nenorimiFiles = {"isvedimas.txt", "vardai_moteru.txt", "vardai_vyru.txt", "temp.txt",
        "Pirmunai1000.txt", "Pirmunai10000.txt", "Pirmunai100000.txt", "Pirmunai1000000.txt", "Pirmunai10000000.txt",
        "Nesimokantys1000.txt", "Nesimokantys10000.txt", "Nesimokantys100000.txt", "Nesimokantys1000000.txt",
        "Nesimokantys10000000.txt"};
00454
00455     f.open("temp.txt");
00456
00457     while (getline(f, eilute))
00458     {
00459
00460         eilute.pop_back();
00461         if (std::find(nenorimiFiles.begin(), nenorimiFiles.end(), eilute) == nenorimiFiles.end())
00462         {
00463             txtfiles.push_back(eilute);
00464         }
00465     }
00466
00467     f.close();
00468     system("del temp.txt");
00469     int a = 1;
00470
00471     for (int i = 0; i < txtfiles.size(); i++)
00472     {
00473         cout < txtfiles[i] < " |" < i + 1 < " pasirinkimas" < endl;
00474         pasirinkimuSUM += 1;
00475     }
00476     cout < "Pasirinkite file: " < endl;
00477     while (true)
00478     {
00479         try
00480         {
00481             cin > pasirinkimas;
00482             if (cin.fail())
00483             {
00484                 cin.clear();
00485                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00486                 throw "Neteisingas pasirinkimas";
00487             }
00488             if (pasirinkimas > pasirinkimuSUM || pasirinkimas < 1)
00489             {
00490                 throw "Neteisingas pasirinkimas";
00491             }
00492             break;
00493         }
00494         catch (const char *masyvas)
00495         {
00496             cout < masyvas < endl;
00497         }
00498     }
00499
00500     return txtfiles[pasirinkimas - 1];
00501 }
00502
00503 bool PalygintiVardas(const Stud &stud1, const Stud &stud2)
00504 {
00505     return stud1.getVardas() < stud2.getVardas();
00506 }
00507
00508 bool PalygintiPavarde(const Stud &stud1, const Stud &stud2)
00509 {
00510     return stud1.getPavarde() < stud2.getPavarde();
00511 }

```

```

00512
00513 bool PalygintiKategorijas(const Stud &stud1, const Stud &stud2){
00514     return stud1.galutinisBalas() > stud2.galutinisBalas();
00515 }
00516
00517 void printCases(vector<Stud> visi, bool outputFILE, int RusiavimasPagal)
00518 {
00519     int SkaiciuotiPagal;
00520     cout << "Galutinis balas skaiciuojamas pagal:" << endl;
00521     cout << "1 - mediana\n";
00522     cout << "2 - vidurki\n";
00523     cin >> SkaiciuotiPagal;
00524     std::ostream *out;
00525     std::ofstream f;
00526     f.open("isvedimas.txt");
00527
00528     if (outputFILE){
00529         out = &f;
00530     }
00531     else{
00532         out = &cout;
00533     }
00534     if (RusiavimasPagal == 1){
00535         sort(visi.begin(), visi.end(), PalygintiVardas);
00536     }
00537     else if (RusiavimasPagal == 2){
00538         sort(visi.begin(), visi.end(), PalygintiPavardes);
00539     }
00540     // pagal mediana
00541     if (SkaiciuotiPagal == 1)
00542     {
00543         if (RusiavimasPagal == 3)
00544         {
00545             sort(visi.begin(), visi.end(), PalygintiKategorijas);
00546         }
00547         *out << std::setw(15) << std::left << "Pavarde" << std::setw(15) << std::left << "Vardas" << std::setw(15) <<
std::left << "Galutinis (Med.)\n";
00548         *out << "-----" << endl;
00549         for (Stud& j : visi)
00550         {
00551             //cout<studentas; => NEVEIKIA GALUTINIO BALO SKAICIAVIMAS
00552             //*out << std::setw(15) << std::left << j.getPavarde() << std::setw(15) << std::left << j.getVardas() <<
std::setw(15) << std::fixed << std::setprecision(2) << j.galutinis() << endl;
00553             *out<j;
00554         }
00555     }
00556     // skaiciuojam rn pagal vidurki
00557     else
00558     {
00559         if (RusiavimasPagal == 3)
00560         {
00561             sort(visi.begin(), visi.end(), PalygintiKategorijas);
00562         }
00563         *out << std::setw(15) << std::left << "Pavarde" << std::setw(15) << std::left << "Vardas" << std::setw(15) <<
std::left << "Galutinis (Vid.)\n";
00564         *out << "-----" << endl;
00565         for (Stud& j : visi)
00566         {
00567             *out << j;
00568             //*out << std::setw(15) << std::left << j.getPavarde() << std::setw(15) << std::left << j.getVardas() <<
std::setw(15) << std::fixed << std::setprecision(2) << j.galutinis() << endl;
00569         }
00570     }
00571 }
00572 void print(vector<Stud> visi, bool outputFILE, int RusiavimasPagal){
00573     std::ostream *out;
00574     std::ofstream f;
00575     f.open("isvedimas.txt");
00576     if (outputFILE)
00577     {
00578         out = &f;
00579     }
00580     else
00581     {
00582         out = &cout;
00583     }
00584     if (RusiavimasPagal == 1)
00585     {
00586         sort(visi.begin(), visi.end(), PalygintiVardas);
00587     }
00588     else if (RusiavimasPagal == 2)
00589     {
00590         sort(visi.begin(), visi.end(), PalygintiPavardes);
00591     }
00592     else if (RusiavimasPagal == 3)
00593     {
00594         sort(visi.begin(), visi.end(), PalygintiKategorijas);

```



```

00595     }
00596     *out < std::setw(15) < std::left < "Pavardė" < std::setw(15) < std::left < "Vardas" < std::setw(15) <
std::left < "Galutinis (Med.)\n";
00597     *out < "-----" < endl;
00598     for (Stud& j : visi)
00599     { *out < j; }
00600 }
00601
00602 double GeneruotiFiles(int StudSkaicius)
00603 {
00604     std::random_device rd;
00605     std::mt19937 mt(rd());
00606     std::uniform_int_distribution<int> dist(1, 10);
00607     string FileName = "Studentai" + std::to_string(StudSkaicius) + ".txt";
00608     std::stringstream BufferisTest;
00609     std::ofstream f;
00610     f.open(FileName);
00611     int pazSk = dist(mt);
00612     auto start = std::chrono::high_resolution_clock::now();
00613     BufferisTest < std::setw(16) < std::left < "Vardas" < std::setw(16) < std::left < "Pavarde" < std::setw(16) <
std::left;
00614     for (int j = 0; j <= pazSk; j++)
00615     {
00616         if (j != pazSk)
00617         {
00618             BufferisTest < "ND" + std::to_string(j + 1) < std::setw(16) < std::left;
00619         }
00620         else
00621             BufferisTest < "EG rez." < endl;
00622     }
00623     for (int i = 1; i <= StudSkaicius; i++)
00624     {
00625         BufferisTest < std::setw(16) < std::left < "Vardas" + std::to_string(i) < std::setw(16) < std::left <
"Pavarde" + std::to_string(i) < std::setw(16) < std::left;
00626         for (int j = 0; j <= pazSk; j++)
00627         {
00628             BufferisTest < dist(mt) < std::setw(16) < std::left;
00629         }
00630         if (i != StudSkaicius)
00631         {
00632             BufferisTest < endl;
00633         }
00634     }
00635     f < BufferisTest.rdbuf();
00636     f.close();
00637     auto end = std::chrono::high_resolution_clock::now();
00638     std::chrono::duration<double> diff = end - start;
00639     return diff.count();
00640 }
00641

```

5.3 functions.h File Reference

```
#include "main.h"
```

Functions

- bool [tinkamas_char](#) (string vardas)
- bool [tinkamas_int](#) (int skaicius)
- void [pazymiu_ivedimas](#) (Stud &studentas, string &ss)
- void [paz_gener](#) (Stud &studentas, string &ss)
- [Stud vardai](#) (Stud &studentas, string &ss)
- void [duomenu_ivedimas](#) (vector< Stud > &studentai, int meniu)
- void [duomenu_generavimas](#) (vector< Stud > &studentai)
- void [printCases](#) (vector< Stud > visi, bool outputFILE, int RusiavimasPagal)
- void [konstruktoriuTest](#) ()
- string [filePasirinkimas](#) ()
- template<typename konteineris>
void [fleskait](#) (konteineris &studentai, bool a, string filePav, double &BendrasLaikas)

- double [GeneruotiFiles](#) (int StudSkaicius)
- bool [PalygintiVardas](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- bool [PalygintiPavardes](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- bool [PalygintiKategorijas](#) (const [Stud](#) &stud1, const [Stud](#) &stud2)
- template<typename konteineris>
void [PrintVektorius](#) (konteineris &nesimokantys, konteineris &pirmunai, int a, int RusiavimasPagal, double &BendrasLaikas)
- template<typename konteineris>
void [vectorIdejimas](#) (int studSkaicius, konteineris &pirmunai, konteineris &nesimokantys, double &BendrasLaikas, konteineris &studentaiTest)
- template<typename konteineris>
void [tyrimai](#) (int pasirinkimasTyrimo, konteineris &pirmunai, konteineris &nesimokantys, konteineris &studentaiTest)

5.3.1 Function Documentation

5.3.1.1 duomenu_generavimas()

```
void duomenu_generavimas (
    vector< Stud > & studentai)
```

Definition at line [429](#) of file [functions.cpp](#).

5.3.1.2 duomenu_ivedimas()

```
void duomenu_ivedimas (
    vector< Stud > & studentai,
    int menu)
```

Definition at line [329](#) of file [functions.cpp](#).

5.3.1.3 filePasirinkimas()

```
string filePasirinkimas ()
```

Definition at line [445](#) of file [functions.cpp](#).

5.3.1.4 fileskait()

```
template<typename konteineris>
void fileskait (
    konteineris & studentai,
    bool a,
    string filePav,
    double & BendrasLaikas)
```

Definition at line [19](#) of file [functions.h](#).

5.3.1.5 GeneruotiFiles()

```
double GeneruotiFiles (  
    int StudSkaicius)
```

Definition at line 602 of file [functions.cpp](#).

5.3.1.6 konstruktoriuTest()

```
void konstruktoriuTest ()
```

Definition at line 156 of file [functions.cpp](#).

5.3.1.7 PalygintiKategorijas()

```
bool PalygintiKategorijas (  
    const Stud & stud1,  
    const Stud & stud2)
```

Definition at line 513 of file [functions.cpp](#).

5.3.1.8 PalygintiPavardes()

```
bool PalygintiPavardes (  
    const Stud & stud1,  
    const Stud & stud2)
```

Definition at line 508 of file [functions.cpp](#).

5.3.1.9 PalygintiVardas()

```
bool PalygintiVardas (  
    const Stud & stud1,  
    const Stud & stud2)
```

Definition at line 503 of file [functions.cpp](#).

5.3.1.10 paz_gener()

```
void paz_gener (  
    Stud & studentas,  
    string & ss)
```

Definition at line 274 of file [functions.cpp](#).

5.3.1.11 pazymiu_ivedimas()

```
void pazymiu_ivedimas (  
    Stud & studentas,  
    string & ss)
```

Definition at line 235 of file [functions.cpp](#).

5.3.1.12 printCases()

```
void printCases (  
    vector< Stud > visi,  
    bool outputFile,  
    int RusiavimasPagal)
```

Definition at line 517 of file [functions.cpp](#).

5.3.1.13 PrintVektorius()

```
template<typename konteineris>  
void PrintVektorius (  
    konteineris & nesimokantys,  
    konteineris & pirmunai,  
    int a,  
    int RusiavimasPagal,  
    double & BendrasLaikas)
```

Definition at line 119 of file [functions.h](#).

5.3.1.14 tinkamas_char()

```
bool tinkamas_char (  
    string vardas)
```

Definition at line 168 of file [functions.cpp](#).

5.3.1.15 tinkamas_int()

```
bool tinkamas_int (  
    int skaicius)
```

Definition at line 180 of file [functions.cpp](#).

5.3.1.16 tyrimai()

```
template<typename konteineris>  
void tyrimai (  
    int pasirinkimasTyrimo,  
    konteineris & pirmunai,  
    konteineris & nesimokantys,  
    konteineris & studentaiTest)
```

Definition at line 259 of file [functions.h](#).

5.3.1.17 vardai()

```
Stud vardai (
    Stud & studentas,
    string & ss)
```

Definition at line 285 of file [functions.cpp](#).

5.3.1.18 vectorIdejimas()

```
template<typename konteineris>
void vectorIdejimas (
    int studSkaicius,
    konteineris & pirmunai,
    konteineris & nesimokantys,
    double & BendrasLaikas,
    konteineris & studentaiTest)
```

Definition at line 225 of file [functions.h](#).

5.4 functions.h

[Go to the documentation of this file.](#)

```
00001
00002 #ifndef FUNCTIONS_H
00003 #define FUNCTIONS_H
00004 #include "main.h"
00005
00006 bool tinkamas_char(string vardas);
00007 bool tinkamas_int(int skaicius);
00008 //double galutinis(Stud studentas, int PagalSkaiciavimas);
00009 void pazymiu_ivedimas(Stud &studentas, string& ss);
00010 void paz_gener(Stud &studentas, string& ss);
00011 Stud vardai(Stud &studentas, string& ss);
00012 void duomenu_ivedimas(vector<Stud> &studentai, int meniu);
00013 void duomenu_generavimas(vector<Stud> &studentai);
00014 void printCases(vector<Stud> visi, bool outputFILE, int RusiavimasPagal);
00015 void konstruktoriuTest();
00016 string filePasirinkimas();
00017
00018 template <typename konteineris>
00019 void fileskait(konteineris &studentai, bool a, string filePav, double &BendrasLaikas)
00020 {
00021     double visasLaikas = 0.0;
00022     vector<string> visaeil;
00023     int pazymys;
00024     string eilute;
00025     string eilute1;
00026     ifstream f;
00027     std::stringstream bufferis;
00028     string testuojamasFile;
00029     if (a == false){
00030         testuojamasFile = filePasirinkimas();
00031     }
00032     if (a == true){
00033         testuojamasFile = filePav;
00034     }
00035     cout << "...\\n";
00036     cout << testuojamasFile << endl;
00037
00038     if (testuojamasFile != "kursiokai.txt"){
00039         testuojamasFile = "Strategijos\\Tyrimo_files\\" + testuojamasFile;
00040     }
00041
00042     while (true)
00043     {
00044         try
00045         {
```

```

00046         f.open(testuojamasFile);
00047         if (!f)
00048         {
00049             throw "Failo atidaryti nepavyko";
00050         }
00051         break;
00052     }
00053     catch (const char *masyvas)
00054     {
00055         cout << masyvas << endl;
00056         testuojamasFile = filePasirinkimas();
00057     }
00058     continue;
00059 }
00060 }
00061 int iteracijos;
00062 (testuojamasFile == "kursiokai.txt") ? iteracijos = 0 : iteracijos = 2;
00063 for (int i = 0; i <= iteracijos; i++)
00064 {
00065     studentai.clear();
00066     bufferis.str("");
00067     bufferis.clear();
00068     auto start = std::chrono::high_resolution_clock::now();
00069     bufferis << f.rdbuf();
00070     f.close();
00071     getline(bufferis, eilute1);
00072     while (bufferis)
00073     {
00074         if (!bufferis.eof())
00075         {
00076             getline(bufferis, eilute);
00077             visaeil.push_back(eilute);
00078         }
00079         else
00080             break;
00081     }
00082     for (string& s : visaeil){
00083         Stud studentas;
00084         std::istringstream f(s);
00085         f >> studentas;
00086         studentai.push_back(studentas);
00087     }
00088     auto end = std::chrono::high_resolution_clock::now();
00089     std::chrono::duration<double> diff = end - start;
00090     if (i != 0){
00091         visasLaikas += diff.count();
00092     }
00093     if (testuojamasFile != "kursiokai.txt" && a == false)
00094     {
00095         cout << i + 1 << " iteracija: " << std::fixed << std::setprecision(3) << diff.count() << "s" << endl;
00096     }
00097 }
00098 if (testuojamasFile != "kursiokai.txt")
00099 {
00100     if (a == false)
00101     {
00102         cout << "Programos vidutinis vykdymo laikas: " << std::fixed << std::setprecision(3) << visasLaikas / 2 <<
00103         "s" << endl;
00104     }
00105     if (a == true)
00106     {
00107         cout << "Nuskaitymas is failo vidutiniskai truko: " << std::fixed << std::setprecision(3) << visasLaikas / 2
00108         << "s" << endl;
00109         BendrasLaikas += (visasLaikas / 2);
00110     }
00111 }
00112 }
00113 double GeneruotiFiles(int StudSkaicius);
00114 bool PalygintiVardas(const Stud &stud1, const Stud &stud2);
00115 bool PalygintiPavardes(const Stud &stud1, const Stud &stud2);
00116 bool PalygintiKategorijas(const Stud &stud1, const Stud &stud2);
00117
00118 template <typename konteineris>
00119 void PrintVektorius(konteineris &nesimokantys, konteineris &pirmunai, int a, int RusiavimasPagal, double
&BendrasLaikas)
00120 {
00121     string FILEMOK = "Pirmunai" + std::to_string(a) + ".txt";
00122     string FILENESIMOK = "Nesimokantys" + std::to_string(a) + ".txt";
00123     std::stringstream buferis;
00124     std::ofstream f, F;
00125     f.open(FILEMOK);
00126
00127     if (RusiavimasPagal == 1)
00128     {
00129         double LaikasRusiavimo = 0.0;

```

```

00130     for (int t = 0; t < 3; t++){
00131     {
00132         auto t1 = std::chrono::high_resolution_clock::now();
00133         if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>){
00134             sort(nesimokantys.begin(), nesimokantys.end(), PalygintiVardas);
00135             sort(pirmunai.begin(), pirmunai.end(), PalygintiVardas);}
00136         else{
00137             nesimokantys.sort(PalygintiVardas);
00138             pirmunai.sort(PalygintiVardas);}
00139
00140         auto t2 = std::chrono::high_resolution_clock::now();
00141         std::chrono::duration<double> difft = t2 - t1;
00142         if (t != 0){
00143             LaikasRusiavimo += difft.count();}
00144     }
00145
00146     cout << "Rusiavimas vidutiniskai uztruko: " << LaikasRusiavimo / 2 << "s" << endl;
00147     BendrasLaikas += (LaikasRusiavimo / 2);
00148 }
00149 else if (RusiavimasPagal == 2){
00150     double LaikasRusiavimo = 0.0;
00151     for (int t = 0; t < 3; t++){
00152         auto t1 = std::chrono::high_resolution_clock::now();
00153         if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>){
00154             sort(nesimokantys.begin(), nesimokantys.end(), PalygintiPavardes);
00155             sort(pirmunai.begin(), pirmunai.end(), PalygintiPavardes);
00156         }
00157         else{
00158             nesimokantys.sort(PalygintiPavardes);
00159             pirmunai.sort(PalygintiPavardes);
00160         }
00161         auto t2 = std::chrono::high_resolution_clock::now();
00162         std::chrono::duration<double> difft = t2 - t1;
00163         if (t != 0){
00164             LaikasRusiavimo += difft.count();}
00165     }
00166     cout << "Rusiavimas vidutiniskai uztruko: " << LaikasRusiavimo / 2 << "s" << endl;
00167     BendrasLaikas += (LaikasRusiavimo / 2);
00168 }
00169 else if (RusiavimasPagal == 3){
00170     double LaikasRusiavimo = 0.0;
00171     for (int t = 0; t < 3; t++){
00172         auto t1 = std::chrono::high_resolution_clock::now();
00173         if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>){
00174             sort(nesimokantys.begin(), nesimokantys.end(), PalygintiKategorijas);
00175             sort(pirmunai.begin(), pirmunai.end(), PalygintiKategorijas);
00176         }
00177         else
00178         {
00179             nesimokantys.sort(PalygintiKategorijas);
00180             pirmunai.sort(PalygintiKategorijas);
00181         }
00182         auto t2 = std::chrono::high_resolution_clock::now();
00183         std::chrono::duration<double> difft = t2 - t1;
00184         if (t != 0){
00185             LaikasRusiavimo += difft.count();}
00186     }
00187 }
00188 cout << "Rusiavimas vidutiniskai uztruko: " << LaikasRusiavimo / 2 << "s" << endl;
00189 BendrasLaikas += (LaikasRusiavimo / 2);
00190 }
00191 else if (RusiavimasPagal == 4){
00192     cout << "Rusiavimas uztruko: 0.000 s" << endl;
00193 }
00194
00195 double IsvedimoLaikas = 0.0;
00196 for (int k = 0; k < 2; k++){
00197     auto start2 = std::chrono::high_resolution_clock::now();
00198     buferis << std::setw(16) << std::left << "Pavarde" << std::setw(16) << std::left << "Vardas" << std::setw(16) <<
std::left << "Galutinis (Vid.)\n";
00199     buferis << "-----" << endl;
00200     for (Stud j : pirmunai)
00201     {
00202         buferis << std::setw(16) << std::left << j.getPavarde() << std::setw(16) << std::left << j.getVardas() <<
std::setw(16) << std::fixed << std::setprecision(2) << j.galutinisBalas() << endl;
00203     }
00204     f << buferis.rdbuf();
00205     buferis.str("");
00206     buferis.clear();
00207     f.close();
00208     pirmunai.clear();
00209     f.open(FILENAMESIMOK);
00210     buferis << std::setw(16) << std::left << "Pavarde" << std::setw(16) << std::left << "Vardas" << std::setw(16) <<
std::left << "Galutinis (Vid.)\n";
00211     buferis << "-----" << endl;
00212     for (Stud j : nesimokantys)
00213     {

```

```

00214         buferis < std::setw(16) < std::left < j.getPavarde() < std::setw(16) < std::left < j.getVardas() <
std::setw(16) < std::fixed < std::setprecision(2) < j.galutinisBalas() < endl;
00215     }
00216     f < buferis.rdbuf();
00217     f.close();
00218     auto end2 = std::chrono::high_resolution_clock::now();
00219     std::chrono::duration<double> diff2 = end2 - start2;
00220 }
00221 nesimokantys.clear();
00222 }
00223
00224 template <typename konteineris>
00225 void vectorIdejimas(int studSkaicius, konteineris &pirmunai, konteineris &nesimokantys, double &BendrasLaikas,
konteineris &studentaiTest)
00226 {
00227     string file = "Studentai" + std::to_string(studSkaicius) + ".txt";
00228     studentaiTest;
00229     fileskait(studentaiTest, true, file, BendrasLaikas);
00230     double visasLaikas1 = 0.0;
00231     for (int c = 0; c < 2; c++)
00232     {
00233         auto start1 = std::chrono::high_resolution_clock::now();
00234         for (Stud& j : studentaiTest)
00235         {
00236             if (j.galutinisBalas() >= 5)
00237             {
00238                 pirmunai.push_back(j);
00239             }
00240             else
00241                 nesimokantys.push_back(j);
00242         }
00243         auto end1 = std::chrono::high_resolution_clock::now();
00244         studentaiTest.clear();
00245         std::chrono::duration<double> diff1 = end1 - start1;
00246         visasLaikas1 += diff1.count();
00247     }
00248     if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>)
00249     {
00250         pirmunai.shrink_to_fit();
00251         nesimokantys.shrink_to_fit();
00252     }
00253
00254     cout < "Duomeni isskirstymas i dvi grupes vidutiniskai trunka: " < visasLaikas1 / 2 < "s\n";
00255     BendrasLaikas += visasLaikas1 / 2;
00256 }
00257
00258 template <typename konteineris>
00259 void tyrimai(int pasirinkimasTyrimo, konteineris &pirmunai, konteineris &nesimokantys, konteineris &studentaiTest)
00260 {
00261     if (pasirinkimasTyrimo == 1)
00262     {
00263         for (int a = 1000; a <= 10000000; a *= 10)
00264         {
00265             cout < "Studentai" < a < ".txt\n";
00266             double laikas = 0.0;
00267             for (int i = 0; i < 2; i++)
00268             {
00269                 double s = GeneruotiFiles(a);
00270                 cout < i + 1 < " iteracija: " < s < "s" < endl;
00271                 laikas += s;
00272             }
00273             cout < "Studentai" < a < ".txt vidutiniskai generuoja: " < laikas / 2 < "s" < endl;
00274             cout < "...n";
00275         }
00276     }
00277     else if (pasirinkimasTyrimo == 2)
00278     {
00279         int RusiavimasPagal;
00280         cout < "Studentus rusiuoti pagal: 1 - vardus, 2 - pavardes, 3 - galutini bala, 4 - duomeni nerusiuoti\n";
00281         while (true){
00282             try
00283             {
00284                 cin > RusiavimasPagal;
00285                 if (cin.fail())
00286                 {
00287                     cin.clear();
00288                     cin.ignore(numeric_limits<streamsize>::max(), '\n');
00289                     throw "Neteisingas pasirinkimas, iveskite skaiciu 1-4";
00290                     break;
00291                 }
00292                 else if (RusiavimasPagal > 4 || RusiavimasPagal < 1){
00293                     throw "Neteisingas pasirinkimas, iveskite skaiciu 1-4";
00294                     break;
00295                 }
00296                 break;
00297             }
00298             catch (const char *masyvas){

```



```

00299         cout << masyvas << endl;}
00300     }
00301     for (int a = 1000; a <= 10000000; a *= 10)
00302     {
00303         double BendrasVidLaikas = 0.0;
00304         vectorIdejimas(a, pirmunai, nesimokantys, BendrasVidLaikas, studentaiTest);
00305         PrintVektorius(nesimokantys, pirmunai, a, RusiavimasPagal, BendrasVidLaikas);
00306         cout << "Visos programos vykdymo laikas: ";
00307         cout << BendrasVidLaikas << "s" << endl;
00308         cout << endl;
00309     }
00310 }
00311 }
00312 #endif

```

5.5 main.cpp File Reference

```
#include "functions.h"
```

Functions

- int [main](#) ()

5.5.1 Function Documentation

5.5.1.1 main()

```
int main ()
```

Definition at line 2 of file [main.cpp](#).

5.6 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include "functions.h"
00002 int main()
00003 {
00004     int meniu;
00005     int paz;
00006     string testavimui;
00007     vector<Stud> visi;
00008     srand(time(NULL));
00009     int rusiavimas;
00010     cout << "Pasirinkite: 1 - duomenys ivedinejami ranka, 2 - pazymiai generuojami, 3 - generuojami visi studentu
duomenys, 4 - duomenys nuskaitomi is failo, 5 - atlikti 1 tyrima/ generuoti files, 6 - atlikti 2 tyrima, 7 - baigti
programa" << endl;
00011     while (true)
00012     {
00013         try
00014         {
00015             cin > meniu;
00016             if (cin.fail())
00017             {
00018                 cin.clear();
00019                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00020                 throw "Neteisingas pasirinkimas, iveskite skaiciu 1-7";
00021                 break;
00022             }
00023             else if (meniu > 7 || meniu < 1)
00024             {
00025                 throw "Neteisingas pasirinkimas, iveskite skaiciu 1-7";
00026                 break;

```

```

00027         }
00028
00029         break;
00030     }
00031     catch (const char *masyvas)
00032     {
00033         cout << masyvas << endl;
00034     }
00035 }
00036
00037 switch (menu)
00038 {
00039 case 1:
00040 {
00041     duomeniu_ivedimas(visi, menu);
00042     printCases(visi, false, 4);
00043     break;
00044 }
00045
00046 case 2:
00047 {
00048     duomeniu_ivedimas(visi, menu);
00049     printCases(visi, false, 4);
00050     break;
00051 }
00052
00053 case 3:
00054 {
00055     duomeniu_generavimas(visi);
00056     printCases(visi, false, 4);
00057     break;
00058 }
00059
00060 case 4:
00061 {
00062     double laik;
00063     int printinimas;
00064     fileskait(visi, false, testavimui, laik);
00065     cout << "Duomenis isvesti terminale - 1, duomenis isvesti faile - 2" << endl;
00066     while (true)
00067     {
00068         try
00069         {
00070             cin > printinimas;
00071             if (cin.fail())
00072             {
00073                 cin.clear();
00074                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00075                 throw "Neteisingas pasirinkimas, iveskite skaiciu 1 arba 2";
00076             }
00077             if (printinimas != 1 && printinimas != 2)
00078             {
00079                 throw "Neteisingas pasirinkimas, iveskite skaiciu 1 arba 2";
00080             }
00081             break;
00082         }
00083         catch (const char *masyvas)
00084         {
00085             cout << masyvas << endl;
00086         }
00087     }
00088     cout << "Duomenys rusiuojami pagal: 1 - vardus; 2 - pavardes; 3 - galutinius balus; 4 - duomenys nera
rusiuojami " << endl;
00089     while (true)
00090     {
00091         try
00092         {
00093             cin > rusiavimas;
00094             if (cin.fail())
00095             {
00096                 cin.clear();
00097                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00098                 throw "Neteisingas pasirinkimas, iveskite skaiciu nuo 1 iki 4";
00099             }
00100             if (rusiavimas > 4 || rusiavimas < 1)
00101             {
00102                 throw "Neteisingas pasirinkimas, iveskite skaiciu nuo 1 iki 4";
00103             }
00104             break;
00105         }
00106         catch (const char *masyvas)
00107         {
00108             cout << masyvas << endl;
00109         }
00110     }
00111     if (printinimas == 1)
00112     {

```

```

00113         printCases(visi, false, rusiavimas);
00114     }
00115     else
00116         printCases(visi, true, rusiavimas);
00117     break;
00118 }
00119 case 5:
00120 {
00121     konteinerisVector pirmunai;
00122     konteinerisVector nesimokantys;
00123     konteinerisVector studentaiTest;
00124     //tyrimai(1, pirmunai, nesimokantys, studentaiTest);
00125     break;
00126 }
00127 case 6:
00128 {
00129     int konteinerioTipas;
00130     cout << "Pasirinkite testuojamo konteinerio tipa:" << endl;
00131     cout << "1 | vector\n"
00132           << "2 | deque\n"
00133           << "3 | list\n";
00134     cin >> konteinerioTipas;
00135     if (konteinerioTipas == 1)
00136     {
00137         konteinerisVector pirmunai;
00138         konteinerisVector nesimokantys;
00139         konteinerisVector studentaiTest;
00140         tyrimai(2, pirmunai, nesimokantys, studentaiTest);
00141     }
00142     if (konteinerioTipas == 2)
00143     {
00144         konteinerisDeque pirmunai;
00145         konteinerisDeque nesimokantys;
00146         konteinerisDeque studentaiTest;
00147         tyrimai(2, pirmunai, nesimokantys, studentaiTest);
00148     }
00149     if (konteinerioTipas == 3)
00150     {
00151         konteinerisList pirmunai;
00152         konteinerisList nesimokantys;
00153         konteinerisList studentaiTest;
00154         tyrimai(2, pirmunai, nesimokantys, studentaiTest);
00155     }
00156     break;
00157 }
00158 }
00159 case 7:
00160 {
00161     return 0;
00162 }
00163 }
00164
00165 }

```

5.7 main.h File Reference

```

#include <iostream>
#include <iomanip>
#include <stdio.h>
#include <stdlib.h>
#include <fstream>
#include <sstream>
#include <vector>
#include <chrono>
#include <cpuid.h>
#include <string>
#include <ctime>
#include <cmath>
#include <algorithm>
#include <cctype>
#include <filesystem>
#include <random>
#include <deque>

```

```
#include <list>
#include <execution>
#include <thread>
```

Classes

- class [zmogus](#)
- class [Stud](#)

Typedefs

- using [konteinerisVector](#) = std::vector<[Stud](#)>
- using [konteinerisList](#) = std::list<[Stud](#)>
- using [konteinerisDeque](#) = std::deque<[Stud](#)>

5.7.1 Typedef Documentation

5.7.1.1 konteinerisDeque

```
using konteinerisDeque = std::deque<Stud>
```

Definition at line [173](#) of file [main.h](#).

5.7.1.2 konteinerisList

```
using konteinerisList = std::list<Stud>
```

Definition at line [172](#) of file [main.h](#).

5.7.1.3 konteinerisVector

```
using konteinerisVector = std::vector<Stud>
```

Definition at line [171](#) of file [main.h](#).

5.8 main.h

[Go to the documentation of this file.](#)

```

00001 #ifndef main_h
00002 #define main_h
00003
00004 #include <iostream>
00005 #include <iomanip>
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <vector>
00011 #include <chrono>
00012 #include <cpuid.h>
00013 #include <string>
00014 #include <ctime>
00015 #include <cmath>
00016 #include <algorithm>
00017 #include <cctype>
00018 #include <filesystem>
00019 #include <random>
00020 #include <filesystem>
00021 #include <deque>
00022 #include <list>
00023 #include <execution>
00024 #include <thread>
00025
00026 using std::cin;
00027 using std::cout;
00028 using std::endl;
00029 using std::getline;
00030 using std::ifstream;
00031 using std::ios;
00032 using std::isalpha;
00033 using std::isdigit;
00034 using std::isspace;
00035 using std::numeric_limits;
00036 using std::sort;
00037 using std::streamsize;
00038 using std::string;
00039 using std::vector;
00040 using std::deque;
00041 using std::list;
00042
00043 class zmogus{
00044     protected:
00045         string vardas;
00046         string pavarde;
00047     public:
00048         zmogus() : vardas(""), pavarde("") {}
00049         zmogus(const string& var, const string& pav) : vardas(var), pavarde(pav) {}
00050         virtual ~zmogus() { vardas = ""; pavarde="";}
00051
00052         virtual void setVardas(const string& var) = 0;
00053         virtual void setPavarde(const string& pav) = 0;
00054
00055         string getVardas()const{return vardas;}
00056         string getPavarde()const{return pavarde;}
00057
00058         //Move constructor
00059         zmogus(zmogus&& z): vardas(std::move(z.vardas)), pavarde(std::move(z.pavarde)) {}
00060
00061         //Move assignment operator
00062         zmogus& operator=(zmogus&& z) {
00063             if (this != &z) {
00064                 vardas = std::move(z.vardas);
00065                 pavarde = std::move(z.pavarde);}
00066             return *this;}
00067 };
00068
00069
00070 class Stud : public zmogus{
00071     private:
00072         //string vardas, pavarde;
00073         int egzaminas;
00074         vector<int> nd;
00075         mutable double BalasGalutinisVid;
00076         mutable bool balasSuskaiciuotas;
00077     public:
00078         Stud() : zmogus("", ""), egzaminas(0), nd{}, BalasGalutinisVid(0), balasSuskaiciuotas(false) {};
00079         //konstruktorius defaultinis
00079         Stud(const string& vrd, const string& pvrd, int egzam, const vector<int>& namud): zmogus(vrd, pvrd),
00080             egzaminas(egzam), nd(namud), BalasGalutinisVid(0), balasSuskaiciuotas(false) {}
00081

```

```

00082      /*Stud(const Stud& studCopy); //copy konstruktorius
00083      Stud& operator=(const Stud& studCopy); //copy assignment
00084      Stud(Stud&& studMove); //move konstruktorius
00085      Stud& operator=(Stud&& studMove); //move assignment */
00086
00088      Stud(const Stud& studCopy) :
00089      zmogus(studCopy.getVardas(), studCopy.getPavarde()),
00090      egzaminas(studCopy.egzaminas),
00091      nd(studCopy.nd),
00092      BalasGalutinisVid(studCopy.BalasGalutinisVid),
00093      balasSuskaiciuotas(studCopy.balasSuskaiciuotas) {}
00094      friend std::ostream& operator<(std::ostream &output, const Stud& studentas);
00095      friend std::istream& operator>(std::istream &input, Stud& studentas);
00096
00098      Stud& operator=(const Stud& studCopy){
00099          if(this != &studCopy){
00100              setVardas(studCopy.getVardas());
00101              setPavarde(studCopy.getPavarde());
00102              nd = studCopy.nd;
00103              egzaminas = studCopy.egzaminas;
00104              BalasGalutinisVid = studCopy.BalasGalutinisVid;
00105              balasSuskaiciuotas = studCopy.balasSuskaiciuotas;}
00106          return *this;
00107      }
00108
00110      Stud(Stud&& studMove) :
00111      zmogus(std::move(studMove)),
00112      nd(std::move(studMove.nd)),
00113      egzaminas(std::move(studMove.egzaminas)),
00114      BalasGalutinisVid(std::move(studMove.BalasGalutinisVid)),
00115      balasSuskaiciuotas(std::move(studMove.balasSuskaiciuotas)){studMove.clear();}
00116
00118      Stud operator=(Stud &&studMove)
00119      {
00120          if (this != &studMove)
00121          {
00122              zmogus::operator=(std::move(studMove));
00123              nd = std::move(studMove.nd);
00124              egzaminas = std::move(studMove.egzaminas);
00125              BalasGalutinisVid = std::move(studMove.BalasGalutinisVid);
00126              balasSuskaiciuotas = std::move(studMove.balasSuskaiciuotas);
00127              studMove.clear();
00128          }
00129          return *this;
00130      }
00131
00132
00133      void clear(){nd.clear(); egzaminas=0; BalasGalutinisVid=0;balasSuskaiciuotas=false;}
00134      ~Stud()override{clear();} //destruktorius
00135
00136      //setteriai
00137      void setVardas(const string& var)override{vardas=var;}
00138      void setPavarde(const string& pav)override{pavarde=pav;}
00139      void setEgzaminas(const int& egz){egzaminas=egz;}
00140      void setBalasGalutinisVid(const double& Balas){BalasGalutinisVid=Balas; balasSuskaiciuotas = true;}
00141      void setND(int paz){nd.push_back(paz);}
00142
00143      //getteriai
00144      int getEgzaminas() const {return egzaminas;}
00145      int getPaz(int i) const {return nd[i];}
00146      const vector<int>& getND() const {return nd;}
00147      double galutinisBalas() const {return BalasGalutinisVid;}
00148      int pazKiekis() const {return nd.size();}
00149      const int& grazintiPaskutini() const {return nd.back();}
00150      void removeLast(){nd.pop_back();}
00151      void removeND(){nd.clear();}
00152
00154      double galutinis() const{
00155          if(!balasSuskaiciuotas){
00156              double suma = 0;
00157              double vidurkis;
00158              if (nd.size() == 1) {
00159                  vidurkis = nd[0];}
00160              else {
00161                  for (size_t ss = 0; ss < nd.size(); ss++) {
00162                      suma += nd[ss];}
00163                  vidurkis = suma / nd.size();}
00164              BalasGalutinisVid = round((0.4 * vidurkis + 0.6 * egzaminas) * 100) / 100;
00165              balasSuskaiciuotas = true;
00166          }
00167          return BalasGalutinisVid;
00168      }
00169 };
00170
00171 using kontainerisVector = std::vector<Stud>;
00172 using kontainerisList = std::list<Stud>;
00173 using kontainerisDeque = std::deque<Stud>;

```

```
00174
00175 #endif
```

5.9 strategija.h File Reference

```
#include "../functions.h"
```

Functions

- `template<typename konteineris>`
`void nuskaitymasFile` (string filePavadinimas, double &visasLaikas, konteineris &studentai)
- `template<typename konteineris>`
`void studentuIsskirstymas` (konteineris &studentai, double &visasLaikas, konteineris &pirmunai, konteineris &nesimokantys)
- `template<typename konteineris>`
`void studentuRusiavimas` (int &rusiavimasPagal, konteineris &pirmunai, konteineris &nesimokantys, double &visasLaikas)
- `template<typename konteineris>`
`void testavimasPrint` (konteineris &studentai, konteineris &pirmunai, konteineris &nesimokantys, int a)
- `template<typename konteineris>`
`void vykdomaPrograma` (int &rusiavimasPagal, konteineris &studentai, konteineris &pirmunai, konteineris &nesimokantys)
- `template<typename konteineris>`
`void studentuRusiavimas2` (int rusiavimasPagal, konteineris &studentai, double &visasLaikas, double &rusiavimoLaikas)
- `template<typename konteineris>`
`void studentuIsskirstymas2` (konteineris &studentai, double &visasLaikas, konteineris &nesimokantys, double &rusiavimoLaikas)
- `template<typename konteineris>`
`void vykdomaPrograma2` (int rusiavimasPagal, konteineris &studentai, konteineris &nesimokantys)
- `template<typename konteineris>`
`void studentuSkirstymas3` (konteineris &studentai, konteineris &nesimokantys, double &visasLaikas)
- `template<typename konteineris>`
`void StudentuRusiavimas3` (konteineris &Studentai, konteineris &nesimokantys, double &visasLaikas, int RusiuotiPagal)
- `template<typename konteineris>`
`void vykdomaPrograma3` (int rusiavimasPagal, konteineris &studentai, konteineris &nesimokantys)

5.9.1 Function Documentation

5.9.1.1 nuskaitymasFile()

```
template<typename konteineris>
void nuskaitymasFile (
    string filePavadinimas,
    double & visasLaikas,
    konteineris & studentai)
```

Definition at line 6 of file [strategija.h](#).

5.9.1.2 studentuIsskirstymas()

```
template<typename konteineris>
void studentuIsskirstymas (
    konteineris & studentai,
    double & visasLaikas,
    konteineris & pirmunai,
    konteineris & nesimokantys)
```

Definition at line 71 of file [strategija.h](#).

5.9.1.3 studentuIsskirstymas2()

```
template<typename konteineris>
void studentuIsskirstymas2 (
    konteineris & studentai,
    double & visasLaikas,
    konteineris & nesimokantys,
    double & rusiavimoLaikas)
```

Definition at line 250 of file [strategija.h](#).

5.9.1.4 studentuRusiavimas()

```
template<typename konteineris>
void studentuRusiavimas (
    int & rusiavimasPagal,
    konteineris & pirmunai,
    konteineris & nesimokantys,
    double & visasLaikas)
```

Definition at line 109 of file [strategija.h](#).

5.9.1.5 studentuRusiavimas2()

```
template<typename konteineris>
void studentuRusiavimas2 (
    int rusiavimasPagal,
    konteineris & studentai,
    double & visasLaikas,
    double & rusiavimoLaikas)
```

Definition at line 234 of file [strategija.h](#).

5.9.1.6 StudentuRusiavimas3()

```
template<typename konteineris>
void StudentuRusiavimas3 (
    konteineris & Studentai,
    konteineris & nesimokantys,
    double & visasLaikas,
    int RusiuotiPagal)
```

Definition at line 327 of file [strategija.h](#).

5.9.1.7 studentuSkirstymas3()

```
template<typename konteineris>
void studentuSkirstymas3 (
    konteineris & studentai,
    konteineris & nesimokantys,
    double & visasLaikas)
```

Definition at line 312 of file [strategija.h](#).

5.9.1.8 testavimasPrint()

```
template<typename konteineris>
void testavimasPrint (
    konteineris & studentai,
    konteineris & pirmunai,
    konteineris & nesimokantys,
    int a)
```

Definition at line 181 of file [strategija.h](#).

5.9.1.9 vykdomaPrograma()

```
template<typename konteineris>
void vykdomaPrograma (
    int & rusiavimasPagal,
    konteineris & studentai,
    konteineris & pirmunai,
    konteineris & nesimokantys)
```

Definition at line 215 of file [strategija.h](#).

5.9.1.10 vykdomaPrograma2()

```
template<typename konteineris>
void vykdomaPrograma2 (
    int rusiavimasPagal,
    konteineris & studentai,
    konteineris & nesimokantys)
```

Definition at line 292 of file [strategija.h](#).

5.9.1.11 vykdomaPrograma3()

```
template<typename konteineris>
void vykdomaPrograma3 (
    int rusiavimasPagal,
    konteineris & studentai,
    konteineris & nesimokantys)
```

Definition at line 418 of file [strategija.h](#).

5.10 strategija.h

[Go to the documentation of this file.](#)

```

00001 #ifndef strategija_h
00002 #define strategija_h
00003 #include "../functions.h"
00004
00005 template <typename konteineris>
00006 void nuskaitymasFile(string filePavadinimas, double &visasLaikas, konteineris &studentai)
00007 {
00008     double laikas = 0.0;
00009     std::vector<string> visaeil;
00010     int pazymys;
00011     string eilute;
00012     string eilute1;
00013     ifstream f;
00014     std::stringstream bufferis;
00015     f.open(filePavadinimas);
00016     for (int i = 0; i < 3; i++)
00017     {
00018         studentai.clear();
00019         bufferis.clear();
00020         bufferis.str("");
00021         auto start = std::chrono::high_resolution_clock::now();
00022         bufferis << f.rdbuf();
00023         f.close();
00024         getline(bufferis, eilute1);
00025         while (bufferis)
00026         {
00027             if (!bufferis.eof())
00028             {
00029                 getline(bufferis, eilute);
00030                 visaeil.push_back(eilute);
00031             }
00032             else
00033                 break;
00034         }
00035
00036         for (const string& s : visaeil)
00037         {
00038             string var, pav;
00039             Stud studentas;
00040             std::istringstream f(s);
00041             f >> var >> pav;
00042             studentas.setVardas(var);
00043             studentas.setPavarde(pav);
00044             while (f >> pazymys)
00045             {
00046                 studentas.setND(pazymys);
00047             }
00048             studentas.setEgzaminas(studentas.grazintiPaskutini());
00049             studentas.removeLast();
00050             studentas.galutinis();
00051             studentai.push_back(studentas);
00052         }
00053
00054         auto end = std::chrono::high_resolution_clock::now();
00055         std::chrono::duration<double> diff = end - start;
00056         if (i != 0)
00057         {
00058             laikas += diff.count();
00059         }
00060     }
00061
00062     visasLaikas += (laikas / 2);
00063     cout << "Failo nuskaitymas vidutiniskai truko: " << laikas / 2 << "s" << endl;
00064     if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>)
00065     {
00066         studentai.shrink_to_fit();
00067     } visaeil.clear();
00068 }
00069
00070 template <typename konteineris>
00071 void studentulsskirstymas(konteineris &studentai, double &visasLaikas, konteineris &pirmunai, konteineris
&nesimokantys)
00072 {
00073     double laikasProgramos = 0.0;
00074
00075     for (int i = 0; i < 3; i++)
00076     {
00077         auto start1 = std::chrono::high_resolution_clock::now();
00078         for (Stud& j : studentai)
00079         {
00080             if (j.galutinisBalas() >= 5)
00081             {

```

```

00082         pirmunai.push_back(j);
00083     }
00084     else
00085         nesimokantys.push_back(j);
00086 }
00087 auto end1 = std::chrono::high_resolution_clock::now();
00088 std::chrono::duration<double> diff1 = end1 - start1;
00089 if (i != 0)
00090 {
00091     laikasProgramos += diff1.count();
00092 }
00093 if (i != 2)
00094 {
00095     pirmunai.clear();
00096     nesimokantys.clear();
00097 }
00098 if constexpr (std::is_same_v<konteineris, vector<Stud> || std::is_same_v<konteineris, deque<Stud>)
00099 {
00100     pirmunai.shrink_to_fit();
00101     nesimokantys.shrink_to_fit();
00102 }
00103 }
00104 visasLaikas += (laikasProgramos / 2);
00105 cout << "Studentu issikirstymas i dvi grupes vidutiniskai truko: " << laikasProgramos / 2 << "s" << endl;
00106 }
00107
00108 template <typename konteineris>
00109 void studentuRusiavimas(int &rusiavimasPagal, konteineris &pirmunai, konteineris &nesimokantys, double &visasLaikas)
00110 {
00111     double time = 0.0;
00112     if constexpr (std::is_same_v<konteineris, std::list<Stud>)
00113     {
00114         for (int i = 0; i < 3; i++)
00115         {
00116             auto start = std::chrono::high_resolution_clock::now();
00117
00118             std::thread t1([&]()
00119             {
00120                 if (rusiavimasPagal == 1) pirmunai.sort(PalygintiVardas);
00121                 else if (rusiavimasPagal == 2) pirmunai.sort(PalygintiPavardes);
00122                 else if (rusiavimasPagal == 3) pirmunai.sort(PalygintiKategorijas);
00123                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00124
00125             std::thread t2([&]()
00126             {
00127                 if (rusiavimasPagal == 1) nesimokantys.sort(PalygintiVardas);
00128                 else if (rusiavimasPagal == 2) nesimokantys.sort(PalygintiPavardes);
00129                 else if (rusiavimasPagal == 3) nesimokantys.sort(PalygintiKategorijas);
00130                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00131
00132             t1.join();
00133             t2.join();
00134
00135             auto end = std::chrono::high_resolution_clock::now();
00136             std::chrono::duration<double> diff = end - start;
00137             if (i != 0)
00138             {
00139                 time += diff.count();
00140             }
00141         }
00142         cout << "Elementu rusiavimas vidutiniskai truko: " << time / 2 << "s" << endl;
00143         visasLaikas += (time / 2);
00144     }
00145     else
00146     {
00147         for (int j = 0; j < 3; j++)
00148         {
00149             auto start = std::chrono::high_resolution_clock::now();
00150
00151             std::thread t1([&]()
00152             {
00153                 if(rusiavimasPagal == 1) std::sort(pirmunai.begin(), pirmunai.end(), PalygintiVardas);
00154                 else if(rusiavimasPagal == 2) std::sort(pirmunai.begin(), pirmunai.end(), PalygintiPavardes);
00155                 else if(rusiavimasPagal == 3) std::sort(pirmunai.begin(), pirmunai.end(), PalygintiKategorijas);
00156                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00157
00158             std::thread t2([&]()
00159             {
00160                 if(rusiavimasPagal == 1) std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiVardas);
00161                 else if(rusiavimasPagal == 2) std::sort(nesimokantys.begin(), nesimokantys.end(),
00162 PalygintiPavardes);
00163                 else if(rusiavimasPagal == 3) std::sort(nesimokantys.begin(), nesimokantys.end(),
00164 PalygintiKategorijas);
00165                 else cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl; });
00166
00167             t1.join();
00168             t2.join();

```

```

00167
00168     auto end = std::chrono::high_resolution_clock::now();
00169     std::chrono::duration<double> difference = end - start;
00170     if (j != 0)
00171     {
00172         time += difference.count();
00173     }
00174 }
00175 cout < "Elementu rusiavimas vidutiniskai truko: " < time / 2 < "s" < endl;
00176 visasLaikas += (time / 2);
00177 }
00178 }
00179
00180 template <typename konteineris>
00181 void testavimasPrint(konteineris &studentai, konteineris &pirmunai, konteineris &nesimokantys, int a)
00182 {
00183     string FILEMOK = "Pirmunai" + std::to_string(a) + ".txt";
00184     string FILENESIMOK = "Nesimokantys" + std::to_string(a) + ".txt";
00185     std::stringstream buferis;
00186     std::ofstream f, F;
00187
00188     f.open(FILEMOK, ios::app);
00189     f.clear();
00190     buferis < std::setw(16) < std::left < "Pavarde" < std::setw(16) < std::left < "Vardas" < std::setw(16) <
std::left < "Galutinis (Vid.)\n";
00191     buferis < "-----" < endl;
00192     for (Stud& j : studentai)//PAKEICIAU IS PIRMUNU I STUDENTUS
00193     {
00194         buferis < std::setw(16) < std::left < j.getPavarde() < std::setw(16) < std::left < j.getVardas() <
std::setw(16) < std::fixed < std::setprecision(2) < j.galutinisBalas() < endl;
00195     }
00196
00197     f < buferis.rdbuf();
00198     buferis.str("");
00199     buferis.clear();
00200     f.close();
00201     F.open(FILENESIMOK, ios::app);
00202     F.clear();
00203     buferis < std::setw(16) < std::left < "Pavarde" < std::setw(16) < std::left < "Vardas" < std::setw(16) <
std::left < "Galutinis (Vid.)\n";
00204     buferis < "-----" < endl;
00205     for (Stud& j : nesimokantys)
00206     {
00207         buferis < std::setw(16) < std::left < j.getPavarde() < std::setw(16) < std::left < j.getVardas() <
std::setw(16) < std::fixed < std::setprecision(2) < j.galutinisBalas() < endl;
00208     }
00209
00210     F < buferis.rdbuf();
00211     F.close();
00212 }
00213
00214 template <typename konteineris>
00215 void vykdomaPrograma(int &rusiavimasPagal, konteineris &studentai, konteineris &pirmunai, konteineris
&nesimokantys){
00216     for (int a = 1000; a <= 10000000; a *= 10)
00217     {
00218         studentai.clear();
00219         nesimokantys.clear();
00220         pirmunai.clear();
00221         double visasLaikas = 0.0;
00222         string testavimoFile = "Tyrimo_files\\Studentai" + std::to_string(a) + ".txt";
00223         cout < "... \n"
00224             < std::to_string(a) + ".txt" < endl;
00225         nuskaitymasFile(testavimoFile, visasLaikas, studentai);
00226         studentuIsskirstymas(studentai, visasLaikas, pirmunai, nesimokantys);
00227         studentuRusiavimas(rusiavimasPagal, pirmunai, nesimokantys, visasLaikas);
00228         // testavimasPrint(studentai, pirmunai, nesimokantys, a);
00229         cout < "Bendras programos vykdymo laikas: " < visasLaikas < "s" < endl;
00230     }
00231 }
00232
00233 template <typename konteineris>
00234 void studentuRusiavimas2(int rusiavimasPagal, konteineris &studentai, double &visasLaikas, double &rusiavimoLaikas)
00235 {
00236     auto start = std::chrono::high_resolution_clock::now();
00237     if constexpr (std::is_same_v<konteineris, std::list<Stud>)
00238         studentai.sort(PalygintiKategorijas);
00239     else
00240         sort(std::execution::par, studentai.begin(), studentai.end(), PalygintiKategorijas);
00241     auto end = std::chrono::high_resolution_clock::now();
00242     std::chrono::duration<double> difference = end - start;
00243
00244     cout < "Studentu rusiuoja: " < difference.count() < endl;
00245     visasLaikas += difference.count();
00246     rusiavimoLaikas += difference.count();
00247 }
00248

```

```

00249 template <typename konteineris>
00250 void studentuIsskirstymas2(konteineris &studentai, double &visasLaikas, konteineris &nesimokantys, double
&rusiavimoLaikas)
00251 {
00252     auto start = std::chrono::high_resolution_clock::now();
00253     auto it = studentai.end();
00254
00255     while (true)
00256     {
00257         --it; // nes pointina ne i galutini elementa, bet i adresa po galutinio elemento
00258         if (it->galutinisBalas() < 5)
00259         {
00260             nesimokantys.push_back(*it);
00261             studentai.pop_back();
00262         }
00263         else
00264             break;
00265     }
00266     auto end = std::chrono::high_resolution_clock::now();
00267     std::chrono::duration<double> difference = end - start;
00268     if constexpr (std::is_same_v<konteineris, std::vector<Stud> ||
std::is_same_v<konteineris, std::deque<Stud>)>
00269     {
00270         studentai.shrink_to_fit();
00271     }
00272     cout << "Studentus vidutiniskai skirste i dvi grupes: " << difference.count() << "s" << endl;
00273     visasLaikas += difference.count();
00274     auto start2 = std::chrono::high_resolution_clock::now();
00275     if constexpr (std::is_same_v<konteineris, std::list<Stud>)>
00276     {
00277         nesimokantys.sort(PalygintiKategorijas);
00278     }
00279     else
00280     {
00281         sort(std::execution::par, nesimokantys.begin(), nesimokantys.end(), PalygintiKategorijas);
00282         auto end2 = std::chrono::high_resolution_clock::now();
00283         std::chrono::duration<double> difference2 = end2 - start2;
00284         cout << "Nesimokanciu konteinerio rusiavimas truko: " << difference2.count() << "s" << endl;
00285         cout << "Bendras rusiavimo laikas: " << rusiavimoLaikas + difference2.count() << "s" << endl;
00286         visasLaikas += difference2.count();
00287     }
00288 }
00289 }
00290
00291 template <typename konteineris>
00292 void vykdomaPrograma2(int rusiavimasPagal, konteineris &studentai, konteineris &nesimokantys)
00293 {
00294     for (int a = 1000; a <= 10000000; a *= 10)
00295     {
00296         studentai.clear();
00297         nesimokantys.clear();
00298         double visasLaikas = 0.0;
00299         string testavimoFile = "Tyrimo_files\\Studentai" + std::to_string(a) + ".txt";
00300         cout << "...\\n"
00301             << std::to_string(a) + ".txt" << endl;
00302         double rusiavimoLaikas = 0.0;
00303         nuskaitymasFile(testavimoFile, visasLaikas, studentai);
00304         studentuRusiavimas2(rusiavimasPagal, studentai, visasLaikas, rusiavimoLaikas);
00305         studentuIsskirstymas2(studentai, visasLaikas, nesimokantys, rusiavimoLaikas);
00306         // testavimasPrint(studentai, nesimokantys, nesimokantys, a);
00307         cout << "Bendras programos vykdymo laikas: " << visasLaikas << "s" << endl;
00308     }
00309 }
00310
00311 template <typename konteineris>
00312 void studentuSkirstymas3(konteineris &studentai, konteineris &nesimokantys, double &visasLaikas)
00313 {
00314     nesimokantys.clear();
00315     auto start = std::chrono::high_resolution_clock::now();
00316     auto it = std::partition(studentai.begin(), studentai.end(), [](Stud &studentas){ return
studentas.galutinisBalas() >= 5; });
00317     nesimokantys.resize(std::distance(it, studentai.end()));
00318     std::copy(it, studentai.end(), nesimokantys.begin());
00319     studentai.erase(it, studentai.end());
00320     auto end = std::chrono::high_resolution_clock::now();
00321     std::chrono::duration<double> diff = end - start;
00322     cout << "Studentu skirstymas i dvi grupes truko: " << diff.count() << "s" << endl;
00323     visasLaikas += diff.count();
00324 }
00325
00326 template <typename konteineris>
00327 void StudentuRusiavimas3(konteineris &Studentai, konteineris &nesimokantys, double &visasLaikas, int RusiuotiPagal)
00328 {
00329     if constexpr (std::is_same_v<konteineris, std::list<Stud>)>
00330     {
00331         auto start = std::chrono::high_resolution_clock::now();
00332
00333         if (RusiuotiPagal == 1)

```

```

00334         Studentai.sort(PalygintiVardas);
00335     else if (RusiuotiPagal == 2)
00336         Studentai.sort(PalygintiPavardes);
00337     else if (RusiuotiPagal == 3)
00338         Studentai.sort(PalygintiKategorijas);
00339     else
00340         cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl;
00341
00342     if (RusiuotiPagal == 1)
00343         nesimokantys.sort(PalygintiVardas);
00344     else if (RusiuotiPagal == 2)
00345         nesimokantys.sort(PalygintiPavardes);
00346     else if (RusiuotiPagal == 3)
00347         nesimokantys.sort(PalygintiKategorijas);
00348     else
00349         cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl;
00350
00351     auto end = std::chrono::high_resolution_clock::now();
00352     std::chrono::duration<double> diff = end - start;
00353     cout << "Studentu rusiavimas uztruko: " << diff.count() << endl;
00354     visasLaikas += diff.count();
00355 }
00356 else
00357 {
00358     auto start = std::chrono::high_resolution_clock::now();
00359
00360     if (RusiuotiPagal == 1)
00361         std::sort(Studentai.begin(), Studentai.end(), PalygintiVardas);
00362     else if (RusiuotiPagal == 2)
00363         std::sort(Studentai.begin(), Studentai.end(), PalygintiPavardes);
00364     else if (RusiuotiPagal == 3)
00365         std::sort(Studentai.begin(), Studentai.end(), PalygintiKategorijas);
00366     else
00367         cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl;
00368
00369     if (RusiuotiPagal == 1)
00370         std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiVardas);
00371     else if (RusiuotiPagal == 2)
00372         std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiPavardes);
00373     else if (RusiuotiPagal == 3)
00374         std::sort(nesimokantys.begin(), nesimokantys.end(), PalygintiKategorijas);
00375     else
00376         cout << "Elementu rusiavimas vidutiniskai truko: 0.000s" << endl;
00377
00378     auto end = std::chrono::high_resolution_clock::now();
00379     std::chrono::duration<double> difference = end - start;
00380     cout << "Studentu rusiavimas uztruko: " << difference.count() << "s" << endl;
00381     visasLaikas += difference.count();
00382 }
00383 }
00384
00385 /*template <typename konteineris>
00386 void print(konteineris& studentai, konteineris& nesimokantys, int rusiavimasPagal, int a){
00387     string FILEMOK = "Pirmunai" + std::to_string(a) + ".txt";
00388     string FILENESIMOK = "Nesimokantys" + std::to_string(a) + ".txt";
00389     std::stringstream buferis;
00390     std::ofstream f, F;
00391
00392     f.open(FILEMOK, ios::app);
00393     f.clear();
00394     buferis << std::setw(16) << std::left << "Pavarde" << std::setw(16) << std::left << "Vardas" << std::setw(16) <<
00395     std::left << "Galutinis (Vid.)\n";
00396     buferis << "-----" << endl;
00397     for (Stud j : studentai)
00398     {
00399         buferis << std::setw(16) << std::left << j.pavarde << std::setw(16) << std::left << j.vardas << std::setw(16) <<
00400         std::fixed << std::setprecision(2) << j.BalasGalutinisVid << endl;
00401     }
00402     f << buferis.rdbuf();
00403     buferis.str("");
00404     buferis.clear();
00405     f.close();
00406     F.open(FILENESIMOK, ios::app);
00407     F.clear();
00408     buferis << std::setw(16) << std::left << "Pavarde" << std::setw(16) << std::left << "Vardas" << std::setw(16) <<
00409     std::left << "Galutinis (Vid.)\n";
00410     buferis << "-----" << endl;
00411     for (Stud j : nesimokantys)
00412     {
00413         buferis << std::setw(16) << std::left << j.pavarde << std::setw(16) << std::left << j.vardas << std::setw(16) <<
00414         std::fixed << std::setprecision(2) << j.BalasGalutinisVid << endl;
00415     }
00416     F << buferis.rdbuf();
00417     F.close();
00418 }*/
00419

```

```

00417 template <typename konteineris>
00418 void vykdomaPrograma3(int rusiavimasPagal, konteineris &studentai, konteineris &nesimokantys)
00419 {
00420     //zmogus asmuo;
00421     for (int a = 100000; a <= 1000000; a *= 10)
00422     {
00423         studentai.clear();
00424         nesimokantys.clear();
00425         double visasLaikas = 0.0;
00426         string testavimoFile = "Tyrimo_files\\Studentai" + std::to_string(a) + ".txt";
00427         cout << "...\\n" << std::to_string(a) + ".txt" << endl;
00428         double rusiavimoLaikas = 0.0;
00429         nuskaitymasFile(testavimoFile, visasLaikas, studentai);
00430         studentuSkirstymas3(studentai, nesimokantys, visasLaikas);
00431         StudentuRusiavimas3(studentai, nesimokantys, visasLaikas, rusiavimasPagal);
00432         //testavimasPrint(studentai, nesimokantys, nesimokantys, a);
00433         //print(studentai, nesimokantys, rusiavimasPagal, a);
00434         cout << "Bendras programos vykdymo laikas: " << visasLaikas << "s" << endl;
00435     }
00436     konstruktoriuTest();
00437 }
00438
00439 #endif

```

5.11 Strategija1.cpp File Reference

```
#include "strategija.h"
```

Functions

- int [main\(\)](#)

5.11.1 Function Documentation

5.11.1.1 main()

```
int main ()
```

Definition at line 5 of file [Strategija1.cpp](#).

5.12 Strategija1.cpp

[Go to the documentation of this file.](#)

```

00001
00002 #include "strategija.h"
00003
00004
00005 int main(){
00006     int konteineriuTipas;
00007     int rusiavimasPagal;
00008     cout << "Programa testuojama su:" << endl;
00009     cout << "1 | vector tipo konteineriu" << endl;
00010     cout << "2 | deque tipo konteineriu" << endl;
00011     cout << "3 | list tipo konteineriu" << endl;
00012     while (true)
00013     {
00014         try
00015         {
00016             cin > konteineriuTipas;
00017             if (cin.fail())
00018             {
00019                 cin.clear();
00020                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00021                 throw "Iveskite skaiciu";
00022                 break;}
00023         }
00024     }
00025 }

```

```

00020         else if (konteineriuTipas > 3 || konteineriuTipas < 1)
00021             {throw "Neteisingas pasirinkimas, iveskite skaiciu 1-3";
00022              break;}
00023         break;
00024     }
00025     catch (const char *masyvas)
00026     {cout << masyvas << endl;}
00027 }
00028
00029 cout << "Studentus rusiuoti pagal:" << endl;
00030 cout << "1 | Vardus\n";
00031 cout << "2 | Pavardes\n";
00032 cout << "3 | Galutini bala\n";
00033 cout << "4 | Duomeni nerusiuoti\n";
00034 while (true)
00035 {   try
00036     {   cin > rusiavimasPagal;
00037         if (cin.fail())
00038         {   cin.clear();
00039             cin.ignore(numeric_limits<streamsize>::max(), '\n');
00040             throw "Iveskite skaiciu";
00041             break;}
00042         else if (rusiavimasPagal > 4 || rusiavimasPagal < 1)
00043             {throw "Neteisingas pasirinkimas, iveskite skaiciu 1-4";
00044              break;}
00045         break;
00046     }
00047     catch (const char *masyvas)
00048     {cout << masyvas << endl;}
00049 }
00050
00051 if (konteineriuTipas == 1) {
00052     konteinerisVector studentai, pirmunai, nesimokantys;
00053     vykdomaPrograma(rusiavimasPagal, studentai, pirmunai, nesimokantys);
00054 }
00055 else if (konteineriuTipas == 2) {
00056     konteinerisDeque studentai, pirmunai, nesimokantys;
00057     vykdomaPrograma(rusiavimasPagal, studentai, pirmunai, nesimokantys);
00058 }
00059 else{
00060     konteinerisList studentai, pirmunai, nesimokantys;
00061     vykdomaPrograma(rusiavimasPagal, studentai, pirmunai, nesimokantys);
00062 }
00063 }

```

5.13 Strategija2.cpp File Reference

```
#include "strategija.h"
```

Functions

- int [main](#) ()

5.13.1 Function Documentation

5.13.1.1 main()

```
int main ()
```

Definition at line 4 of file [Strategija2.cpp](#).

5.14 Strategija2.cpp

[Go to the documentation of this file.](#)

```

00001
00002 #include "strategija.h"
00003
00004 int main(){
00005     int konteineriuTipas;
00006     int rusiavimasPagal;
00007     cout << "Programa testuojama su:\n";
00008     cout << "1 | vector tipo konteineriu\n";
00009     cout << "2 | deque tipo konteineriu\n";
00010     cout << "3 | list tipo konteineriu\n";
00011     while (true)
00012     {
00013         try
00014         {
00015             cin > konteineriuTipas;
00016             if (cin.fail())
00017             {
00018                 cin.clear();
00019                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00020                 throw "Iveskite skaiciu";
00021                 break;}
00022             else if (konteineriuTipas > 3 || konteineriuTipas < 1)
00023             {throw "Neteisingas pasirinkimas, iveskite skaiciu 1-3";
00024                 break;}
00025             break;
00026         }
00027         catch (const char *masyvas)
00028         {cout << masyvas << endl;}
00029     }
00030     cout << "Studentus rusiuoti pagal:\n";
00031     cout << "1 | Vardus\n";
00032     cout << "2 | Pavardes\n";
00033     cout << "3 | Galutini bala\n";
00034     cout << "4 | Duomeniu nerusiuoti\n";
00035     while (true)
00036     {
00037         try
00038         {
00039             cin > rusiavimasPagal;
00040             if (cin.fail())
00041             {
00042                 cin.clear();
00043                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00044                 throw "Iveskite skaiciu";
00045                 break;}
00046             else if (rusiavimasPagal > 4 || rusiavimasPagal < 1)
00047             {throw "Neteisingas pasirinkimas, iveskite skaiciu 1-4";
00048                 break;}
00049             break;
00050         }
00051         catch (const char *masyvas)
00052         {cout << masyvas << endl;}
00053     }
00054     if (konteineriuTipas == 1) {
00055         konteinerisVector studentai, nesimokantys;
00056         vykdomaPrograma2(rusiavimasPagal, studentai, nesimokantys);
00057     }
00058     else if (konteineriuTipas == 2) {
00059         konteinerisDeque studentai, nesimokantys;
00060         vykdomaPrograma2(rusiavimasPagal, studentai, nesimokantys);
00061     }
00062     else{
00063         konteinerisList studentai, nesimokantys;
00064         vykdomaPrograma2(rusiavimasPagal, studentai, nesimokantys);
00065     }
00066 }
00067 }
```

5.15 Strategija3.cpp File Reference

```
#include "strategija.h"
```

Functions

- `int main ()`

5.15.1 Function Documentation

5.15.1.1 main()

```
int main ()
```

Definition at line 4 of file [Strategija3.cpp](#).

5.16 Strategija3.cpp

[Go to the documentation of this file.](#)

```
00001
00002 #include "strategija.h"
00003
00004 int main(){
00005     int konteineriuTipas;
00006     int rusiavimasPagal;
00007     cout << "Programa testuojama su:\n";
00008     cout << "1 | vector tipo konteineriu\n";
00009     cout << "2 | deque tipo konteineriu\n";
00010     cout << "3 | list tipo konteineriu\n";
00011     while (true)
00012     {
00013         try
00014         {
00015             cin > konteineriuTipas;
00016             if (cin.fail())
00017             {
00018                 cin.clear();
00019                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00020                 throw "Iveskite skaiciu";
00021                 break;
00022             }
00023             else if (konteineriuTipas > 3 || konteineriuTipas < 1)
00024             {
00025                 throw "Neteisingas pasirinkimas, iveskite skaiciu 1-3";
00026                 break;
00027             }
00028             catch (const char *masyvas)
00029             {
00030                 cout << masyvas << endl;
00031             }
00032             cout << "Studentus rusiuoti pagal:\n";
00033             cout << "1 | Vardus\n";
00034             cout << "2 | Pavardes\n";
00035             cout << "3 | Galutini bala\n";
00036             cout << "4 | Duomeniu nerusiuoti\n";
00037             while (true)
00038             {
00039                 try
00040                 {
00041                     cin > rusiavimasPagal;
00042                     if (cin.fail())
00043                     {
00044                         cin.clear();
00045                         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00046                         throw "Iveskite skaiciu";
00047                         break;
00048                     }
00049                     else if (rusiavimasPagal > 4 || rusiavimasPagal < 1)
00050                     {
00051                         throw "Neteisingas pasirinkimas, iveskite skaiciu 1-4";
00052                         break;
00053                     }
00054                     break;
00055                 }
00056                 catch (const char *masyvas)
00057                 {
00058                     cout << masyvas << endl;
00059                 }
00060             }
00061             if (konteineriuTipas == 1) {
00062                 konteinerisVector studentai, nesimokantys;
00063                 vykdomaPrograma3(rusiavimasPagal, studentai, nesimokantys);
00064             }
00065             else if (konteineriuTipas == 2) {
00066                 konteinerisDeque studentai, nesimokantys;
00067                 vykdomaPrograma3(rusiavimasPagal, studentai, nesimokantys);
00068             }
00069             else{
00070                 konteinerisList studentai, nesimokantys;
00071                 vykdomaPrograma3(rusiavimasPagal, studentai, nesimokantys);
00072             }
00073         }
00074     }
```

Index

- ~Stud
 - Stud, [9](#)
- ~zmogus
 - zmogus, [13](#)
- clear
 - Stud, [9](#)
- constrTest
 - functions.cpp, [16](#)
- copyAssignTest
 - functions.cpp, [16](#)
- copyConstrTest
 - functions.cpp, [16](#)
- destructorTest
 - functions.cpp, [16](#)
- duomenu_generavimas
 - functions.cpp, [16](#)
 - functions.h, [28](#)
- duomenu_ivedimas
 - functions.cpp, [16](#)
 - functions.h, [28](#)
- filePasirinkimas
 - functions.cpp, [16](#)
 - functions.h, [28](#)
- fileskait
 - functions.h, [28](#)
- functions.cpp, [15](#)
 - constrTest, [16](#)
 - copyAssignTest, [16](#)
 - copyConstrTest, [16](#)
 - destructorTest, [16](#)
 - duomenu_generavimas, [16](#)
 - duomenu_ivedimas, [16](#)
 - filePasirinkimas, [16](#)
 - GeneruotiFiles, [16](#)
 - inputTest, [17](#)
 - konstruktoriuTest, [17](#)
 - moveAssignTest, [17](#)
 - moveConstTest, [17](#)
 - operator<<, [17](#)
 - operator>>, [17](#)
 - outputTest, [17](#)
 - PalygintiKategorijas, [18](#)
 - PalygintiPavardes, [18](#)
 - PalygintiVardas, [18](#)
 - paz_gener, [18](#)
 - pazymiu_ivedimas, [18](#)
 - print, [18](#)
 - printCases, [19](#)
 - tikrinimas, [19](#)
 - tinkamas_char, [19](#)
 - tinkamas_int, [19](#)
 - vardai, [19](#)
- functions.h, [27](#)
 - duomenu_generavimas, [28](#)
 - duomenu_ivedimas, [28](#)
 - filePasirinkimas, [28](#)
 - fileskait, [28](#)
 - GeneruotiFiles, [28](#)
 - konstruktoriuTest, [29](#)
 - PalygintiKategorijas, [29](#)
 - PalygintiPavardes, [29](#)
 - PalygintiVardas, [29](#)
 - paz_gener, [29](#)
 - pazymiu_ivedimas, [29](#)
 - printCases, [30](#)
 - PrintVektorius, [30](#)
 - tinkamas_char, [30](#)
 - tinkamas_int, [30](#)
 - tyrimai, [30](#)
 - vardai, [30](#)
 - vectorIdejimas, [31](#)
- galutinis
 - Stud, [9](#)
- galutinisBalas
 - Stud, [9](#)
- GeneruotiFiles
 - functions.cpp, [16](#)
 - functions.h, [28](#)
- getEgzaminas
 - Stud, [9](#)
- getND
 - Stud, [10](#)
- getPavarde
 - zmogus, [13](#)
- getPaz
 - Stud, [10](#)
- getVaras
 - zmogus, [13](#)
- grazintiPaskutini
 - Stud, [10](#)
- inputTest
 - functions.cpp, [17](#)
- konstruktoriuTest
 - functions.cpp, [17](#)

- functions.h, 29
- konteinerisDeque
 - main.h, 38
- konteinerisList
 - main.h, 38
- konteinerisVector
 - main.h, 38
- main
 - main.cpp, 35
 - Strategija1.cpp, 49
 - Strategija2.cpp, 50
 - Strategija3.cpp, 52
- main.cpp, 35
 - main, 35
- main.h, 37
 - konteinerisDeque, 38
 - konteinerisList, 38
 - konteinerisVector, 38
- moveAssignTest
 - functions.cpp, 17
- moveConstTest
 - functions.cpp, 17
- nuskaitymasFile
 - strategija.h, 41
- operator<<
 - functions.cpp, 17
 - Stud, 12
- operator>>
 - functions.cpp, 17
 - Stud, 12
- operator=
 - Stud, 10
 - zmogus, 14
- outputTest
 - functions.cpp, 17
- PalygintiKategorijas
 - functions.cpp, 18
 - functions.h, 29
- PalygintiPavarde
 - functions.cpp, 18
 - functions.h, 29
- PalygintiVardas
 - functions.cpp, 18
 - functions.h, 29
- pavarde
 - zmogus, 14
- paz_gener
 - functions.cpp, 18
 - functions.h, 29
- pazKiekis
 - Stud, 10
- pazymiu_ivedimas
 - functions.cpp, 18
 - functions.h, 29
- print
 - functions.cpp, 18
- printCases
 - functions.cpp, 19
 - functions.h, 30
- PrintVektorius
 - functions.h, 30
- removeLast
 - Stud, 10
- removeND
 - Stud, 11
- setBalasGalutinisVid
 - Stud, 11
- setEgzaminas
 - Stud, 11
- setND
 - Stud, 11
- setPavarde
 - Stud, 11
 - zmogus, 14
- setVardas
 - Stud, 11
 - zmogus, 14
- strategija.h, 41
 - nuskaitymasFile, 41
 - studentuIsskirstymas, 41
 - studentuIsskirstymas2, 42
 - studentuRusiavimas, 42
 - studentuRusiavimas2, 42
 - StudentuRusiavimas3, 42
 - studentuSkirstymas3, 42
 - testavimasPrint, 43
 - vykdomaPrograma, 43
 - vykdomaPrograma2, 43
 - vykdomaPrograma3, 43
- Strategija1.cpp, 49
 - main, 49
- Strategija2.cpp, 50
 - main, 50
- Strategija3.cpp, 51
 - main, 52
- Stud, 7
 - ~Stud, 9
 - clear, 9
 - galutinis, 9
 - galutinisBalas, 9
 - getEgzaminas, 9
 - getND, 10
 - getPaz, 10
 - grazintiPaskutini, 10
 - operator<<, 12
 - operator>>, 12
 - operator=, 10
 - pazKiekis, 10
 - removeLast, 10
 - removeND, 11
 - setBalasGalutinisVid, 11
 - setEgzaminas, 11

- setND, [11](#)
- setPavarde, [11](#)
- setVardas, [11](#)
- Stud, [8](#), [9](#)
- studentuIsskirstymas
 - strategija.h, [41](#)
- studentuIsskirstymas2
 - strategija.h, [42](#)
- studentuRusiavimas
 - strategija.h, [42](#)
- studentuRusiavimas2
 - strategija.h, [42](#)
- StudentuRusiavimas3
 - strategija.h, [42](#)
- studentuSkirstymas3
 - strategija.h, [42](#)
- testavimasPrint
 - strategija.h, [43](#)
- tikrinimas
 - functions.cpp, [19](#)
- tinkamas_char
 - functions.cpp, [19](#)
 - functions.h, [30](#)
- tinkamas_int
 - functions.cpp, [19](#)
 - functions.h, [30](#)
- tyrimai
 - functions.h, [30](#)
- vardai
 - functions.cpp, [19](#)
 - functions.h, [30](#)
- vardas
 - zmogus, [14](#)
- vectorIdejimas
 - functions.h, [31](#)
- vykdomaPrograma
 - strategija.h, [43](#)
- vykdomaPrograma2
 - strategija.h, [43](#)
- vykdomaPrograma3
 - strategija.h, [43](#)
- zmogus, [12](#)
 - ~zmogus, [13](#)
 - getPavarde, [13](#)
 - getVardas, [13](#)
 - operator=, [14](#)
 - pavarde, [14](#)
 - setPavarde, [14](#)
 - setVardas, [14](#)
 - vardas, [14](#)
 - zmogus, [13](#)